

LabDaemon Desktop

LabDaemon is a standalone Windows and Linux application for school lab instruments. A small Qt-free core runs three layers of plugins: transports, devices and experiments. A PySide6 interface on top offers a guided mode for first runs and students, and an expert mode for everything after.

Replaces the ipywidgets apps in `labintfactory` 0.1.1

Stack Python ≥ 3.14 · PySide6 · pyqtgraph · numpy · pyserial Date 28 September 2026

How to review. Settled choices carry a **DECISION** label. All open questions are now answered; §14 lists what to verify first, in M0. In the mockups (§8), the numbered markers 1 refer to the notes under each figure.

What changed in draft 0.6: hardware information for every device.

- **Hardware sheets** (§7.4, mockup 9). Each device plugin ships its parts, key datasheet figures, charts, notes and datasheet links. A new *Hardware* tab in the Devices view shows them next to a live operating point: how close each channel is to saturating, the resolution, and what a setting change would do.
- **The code uses the sheet's figures too.** Writing the TSL2591 sheet exposed a bug: the current code checks saturation against the TSL2561's 65 000 counts, but at 100 ms the TSL2591 tops out at 36 863, so saturation is never detected there. It is now in the bug table (§9).
- An optional `HW?` command lets a board confirm its parts, so one plugin can support hardware variants (§4.4).
- The example readings throughout the document now agree with the TSL2591 lux formula (for example 5 691 / 1 899 counts → 412.4 lx at 100 ms, gain 25×).
- Draft 0.5 added the Firmware view with the Monaco editor (§7.3, mockup 8).

0 Decisions already taken

TOPIC	DECISION	CONSEQUENCE FOR THE DESIGN
GUI toolkit	PySide6 (Qt 6) + pyqtgraph	Qt imports live only in <code>gui/</code> and in optional <code>gui.py</code> modules of plugins.
Architecture	Core + three plugin layers	Transports → devices → experiments. Experiments see devices only through <i>capabilities</i> .
Discovery	Entry points for built-in plugins, a folder for development and unpackaged plugins	Both routes use the same mapping format and the same loader (§5).

TOPIC	DECISION	CONSEQUENCE FOR THE DESIGN
Concurrency	One worker thread per device	Precisely, one worker per <i>connection</i> : a board, or a Vernier sensor. Each connection is touched by exactly one thread, and everyone else talks to it through a queue. Functions on a combined board share their board's worker (§4.5).
Repository	Option A: one repo, plugins inside	One package, one version, and entry points declared in the project's own <code>pyproject.toml</code> .
Platforms	Windows and Linux	platformdirs for paths, safe file names, serial-port identity by USB serial number, installers for both (§11).
Users	Mostly teachers, sometimes students	Guided mode on first run, expert mode afterwards, plus a lockable student mode (§6).
Jupyter	The core stays usable from Jupyter; the ipywidgets UI is retired	A <code>Lab</code> facade in the core, and no Qt import anywhere below the GUI (§10).
Firmware Q1	Firmware can change; other microcontrollers will be used	One board-independent protocol for all boards (LabInt wire protocol), with a C++ reference library. Legacy firmware is not supported (§4.4).
Stopping Q2	Steppers can stop mid-move	Long operations are asynchronous and <code>STOP</code> is always accepted; Stop and Esc act immediately (§4.5).
Analysis Q6	scipy is a runtime dependency	Non-linear fits with uncertainties through <code>scipy.optimize.curve_fit</code> , and a shared model library (§4.7).
Languages Q4	English by default, Italian available	English source strings; Italian catalogs for the GUI, the core and built-in plugins; a completeness check in CI (§7.1).
Boards Q3	Combined boards are possible	A board reports its <code>functions</code> ; each function is a device with its own plugin, capabilities and lease (§4.3).
Microcontrollers Q12	Arduino (AVR), ESP32, STM32, Teensy, all in C++	One C++ <code>LabInt</code> library on the Arduino core API, with platform adapters for storage and unique IDs. Built with PlatformIO. No MicroPython module.
Links Q13, Q7	ESP32 over USB, Wi-Fi or Bluetooth; Vernier over USB or Bluetooth; chosen by the user	Transports for serial, TCP (Wi-Fi) and BLE carry the same protocol. Discovery covers all three links, and each board remembers its preferred link (§4.4).
Student lock Q5	Implemented	A PIN, re-locking after inactivity, and a machine policy that can force it at startup (§6.1).
Sessions Q8	One <code>.labint</code> zip per session	Format as in §4.7.
Name Q9	LabDaemon	App, CLI, folders and the Python package are named <code>labdaemon</code> . The protocol and firmware library stay "LabInt".
Installation Q10	For all users (central) or per user	Dual-mode Windows installer with silent switches, a .deb and an AppImage on Linux, and a machine policy file (§11).
Several experiments Q11	Tabs with device leases	Leases are per function. Watching a device doesn't need a lease; controlling it does (§7.2).
Python	3.14 or newer	<code>requires-python = ">=3.14"</code> ; CI and installers use 3.14 (§11.3).
Wi-Fi addressing Q14	Lab PCs reach the ESP32 boards over Wi-Fi; on another network the user gives the IP address	mDNS discovery on the PC's own network, and a saved IP address otherwise. Optional static IP in the firmware (§4.4).
Package Q15	<code>labdaemon</code> , not on PyPI	Distributed as installers and release wheels; the repo can be installed directly for Jupyter use.
Windows installer Q16	Silent EXE now, maybe MSI later	Inno Setup now. An MSI would wrap the same PyInstaller output (§11.1).
Firmware tools	Update, compile, upload and edit firmware from LabDaemon; the editor is Monaco	<code>arduino-cli</code> from Arduino IDE 2 (or standalone); Monaco through <code>qtmonaco</code> ; the Arduino language server for completion (§7.3).

TOPIC	DECISION	CONSEQUENCE FOR THE DESIGN
Hardware sheets	Every device shows its hardware information and datasheet figures	A <code>hardware.toml</code> and notes per plugin; a Hardware tab with a live operating point; the plugin code reads the same figures (§7.4).

1 Goals and non-goals

Goals

- Parity first: the photometer (absorbance, kinetics) and magnetic-field-profile labs run on real hardware on both platforms.
- A new device or experiment means adding one plugin. No edits to the core or the GUI.
- Any microcontroller whose firmware follows the LabInt wire protocol can be connected; only a device plugin is needed on the PC side.
- The UI never freezes, and no call can hang on a silent device: every I/O operation has a deadline.
- Every measurement is saved with its context (devices, settings, calibration, times) and can be reopened later.
- The same experiment logic runs in the GUI, in Jupyter and in tests with simulated devices.

Non-goals for v1

- Networked or remote acquisition (the old option C). The core keeps it possible later.
- High-rate acquisition. The target is ≤ 100 samples/s per device; buffered streaming can come later.
- Real access control. Student mode is a convenience lock, not security.
- Plugin sandboxing. Plugins are trusted Python code.

2 Architecture

Front-ends sit on top of a core that owns every cross-cutting service. Experiments depend on the core and on capability interfaces. Devices implement those interfaces on top of transports. The dashed line is the Qt boundary: nothing below it imports PySide6.

FRONT-ENDS

gui · PySide6 + pyqtgraph

Jupyter / scripts

pytest

Qt boundary

no Qt below this line (checked by a test)

EXPERIMENT PLUGINS

logic, wizard, optional panel

absorbance

kinetics

field_profile

data_logger

+ plugin folder

CORE

contracts & services

capabilities

parameters

registry

boards & device manager

leases

policy

workers

experiment runtime

data & sessions

calibrations

analysis

events

settings

DEVICE PLUGINS

implement capabilities

tsl2591 photometer

stepper actuator

thermometer

vernier go direct

simulated

+ plugin folder

TRANSPORTS

bytes and framing

serial · USB

tcp · Wi-Fi + mDNS

ble · Bluetooth LE

LabInt wire protocol

godirect · USB / BLE

simulated

USB · Wi-Fi · BLE

PC above · microcontroller below

FIRMWARE

runs on the boards

protocol spec

LabInt C++ library

links: Serial · Wi-Fi · BLE

AVR · ESP32 · STM32 · Teensy

reference sketches

Dependency rules

- R1** `labdaemon.core` never imports `gui`, `PySide6` or `pyqtgraph`. A test enforces this by importing the core with Qt blocked.
- R2** Plugins depend on the core only. A plugin's optional `gui.py` may import Qt and `labdaemon.gui.widgets`. The core loads it lazily, by name, only inside the GUI.
- R3** The GUI never calls a device or transport directly. It goes through the device manager's handles, so every call lands on that device's worker thread.
- R4** Experiments reach devices only through capabilities (`LightSensor`, `LinearActuator`, ...), never through a concrete device class.
- R5** Plugins do not import each other. Code two plugins need goes into the core, for example `core.analysis.linear_fit`.

3 Repository layout

This is option A: one package with the built-in plugins inside it, each in its own sub-package, so any of them can later be split into a separate distribution without code changes.

```

labdaemon/
├─ pyproject.toml          # deps, optional extras, entry points of built-in plugins
├─ src/labdaemon/
│   └─ __init__.py        # exposes Lab (script/Jupyter facade) and __version__
│   └─ __main__.py        # python -m labdaemon → starts the GUI
│   └─ core/              # — no Qt anywhere in here —
│       ├── capabilities.py # Sensor, LightSensor, LightSource, LinearActuator, MagneticFieldSensor
│       ├── parameters.py  # Parameter, ParameterSet: typed values, limits, validation
│       ├── device.py      # Device base class (one board function), DeviceInfo, DeviceState, Command
│       ├── board.py       # Board: one connection, its functions, link preference
│       ├── transport.py   # Transport base class, exception hierarchy
│       ├── worker.py      # BoardWorker (1 thread per connection), Operation, DeviceProxy
│       ├── manager.py     # DeviceManager: discovery on all links, connect, leases, reconnect
│       ├── policy.py      # machine policy + user settings merge, student lock
│       ├── experiment.py  # Experiment base, Requirement, @action, @run, RunContext
│       ├── wizard.py      # declarative wizard steps (pure data)
│       ├── data.py        # Channel, Dataset
│       ├── session.py     # Session and the .labint file format
│       ├── calibration.py # Calibration, CalibrationLibrary
│       ├── analysis.py    # fits with uncertainties (scipy), model library
│       ├── i18n.py        # gettext setup, per-plugin catalogs, _() and N__()
│       ├── registry.py    # plugin discovery: entry points + plugin folders
│       ├── events.py      # small thread-safe Signal
│       ├── settings.py    # user settings (TOML) and platformdirs paths
│       ├── firmware.py    # toolchain discovery, compile, upload, board-type table, templates
│       └─ hardware.py     # HardwareSheet, Component, OperatingValue: loads hardware.toml, charts,
notes
├─ lab.py                 # Lab facade
├─ transports/
│   ├── serial.py         # SerialTransport (pyserial), port listing with VID:PID and serial no.
│   ├── tcp.py            # TcpTransport + mDNS discovery (zeroconf) for Wi-Fi boards
│   ├── ble.py            # BleTransport: Nordic UART Service over bleak
│   ├── wire.py           # WireClient: LabInt wire protocol (commands, replies, events)
│   └─ simulated.py       # SimulatedTransport + WireSimulator (the spec, executable)
├─ devices/
│   ├── tsl2591/          # __init__.py device.py lux.py simulated.py gui.py hardware/
│   ├── stepper/          # device.py simulated.py gui.py (jog panel)
│   ├── thermometer/     # example of a second function on a combined board
│   └─ vernier/           # device.py (USB or BLE via godirect) simulated.py
├─ experiments/
│   ├── absorbance/       # experiment.py wizard.py gui.py assets/
│   ├── kinetics/
│   ├── field_profile/
│   └─ data_logger/
├─ locale/it/LC_MESSAGES/ # labdaemon.po (core); each plugin carries its own locale/
├─ gui/
│   ├── app.py            # QApplication, theme, translations, logging, main()
│   ├── main_window.py
│   ├── bridge.py         # core Signal / Future → queued Qt signal
│   ├── views/
│   ├── editor/
│   ├── experiment_host.py # hosts the guided or expert workspace of one experiment
│   ├── wizard/
│   ├── widgets/
│   └─ i18n/              # ParameterForm LivePlot DatasetTable LogView DeviceChip StepRail
├─ tests/                 # labdaemon_it.ts (source) → .qm (built)
├─ docs/                  # core/ transports/ devices/ experiments/ gui/ architecture/
├─ firmware/              # user guide, plugin author guide (English, Italian)
│   ├── PROTOCOL.md       # LabInt wire protocol spec, versioned
│   ├── lib/LabInt/
│   │   └─ src/platform/  # C++ library: parser, standard commands, functions, events, links
│   ├── sketches/         # avr.cpp esp32.cpp stm32.cpp teensy.cpp: ID, storage
│   └─ test/              # photometer/ stepper/ combo-esp32/ ... each with platformio.ini
├─ packaging/             # parser unit tests (PlatformIO "native" environment)
└─ template               # PyInstaller spec, Inno Setup script, .deb, AppImage, udev rules, policy

```

4 Core

4.1 Capabilities

Capabilities are small `typing.Protocol` interfaces with typed results. An experiment declares the capabilities it needs; a device declares the ones it provides. One device can provide several: the photometer board is both a `LightSensor` and a `LightSource`. The same absorbance experiment would then work with a Vernier light sensor, or with a simulated one.

```
# core/capabilities.py (abridged)
@dataclass(frozen=True)
class LightReading:
    lux: float
    broadband: int          # raw full-spectrum counts
    infrared: int           # raw IR counts
    saturated: bool

@runtime_checkable
class Sensor(Protocol):
    """Generic view that every sensor offers: named channels with units."""
    def channels(self) -> list[Channel]: ...
    def read(self) -> dict[str, float]: ...          # channel key -> value

class LightSensor(Protocol):
    def read_light(self, samples: int = 1) -> LightReading: ...

class LightSource(Protocol):
    def colors(self) -> list[str]: ...
    def power_range(self, color: str) -> tuple[int, int]: ...
    def set_output(self, color: str, power: int) -> None: ...

class LinearActuator(Protocol):
    travel_mm: float
    def position_mm(self) -> float: ...
    def move_to(self, mm: float) -> Operation: ...      # via a proxy: returns when reached; cancellable
    def home(self) -> None: ...

class MagneticFieldSensor(Protocol):
    def read_field(self, window_ms: int, samples: int) -> FieldReading: ...  # bx, by, bz in mT
```

CAPABILITY	PROVIDED BY (V1)	USED BY
<code>Sensor</code>	every sensor device	data logger, device live view, generic plots
<code>LightSensor</code>	tsl2591 photometer, simulated	absorbance, kinetics
<code>LightSource</code>	tsl2591 photometer (LED board), simulated	absorbance, kinetics
<code>LinearActuator</code>	stepper actuator, simulated	field profile
<code>MagneticFieldSensor</code>	Vernier Go Direct 3D magnetic (USB or Bluetooth), simulated	field profile
<code>TemperatureSensor</code>	thermometer function (for example on a combined ESP32), Vernier temperature, simulated	data logger; kinetics (optional temperature channel)

Capabilities only grow by adding methods that have default implementations. Anything else is a breaking change and bumps the plugin API version (§5).

4.2 Parameters

Devices and experiments describe their settings as `Parameter` objects. The GUI generates forms from them, wizards pick subsets of them, and Jupyter users get validation for free. A `ParameterSet` holds the current values and is the **single source of truth**: widgets bind to it and never keep a copy of their own. This removes the "slider shows one value, code uses another" class of bugs found in the current code.

```
@dataclass(frozen=True)
class Parameter:
    key: str          # "led_power"
    label: str        # translatable: _("LED power")
    type: type        # int | float | str | bool
    default: Any
    unit: str | None = None  # "ms", "mm", "mol/L"
    choices: Sequence | None = None  # e.g. gain: (1, 25, 428, 9876)
    minimum: float | None = None
    maximum: float | None = None
    step: float | None = None
    advanced: bool = False  # hidden in guided and student mode
    help: str = ""
```

Limits can depend on other values. The allowed `led_power` range depends on `led_color`, so after every change the device recomputes `parameters()` and the form updates only the affected field. Out-of-range values are rejected with a message rather than silently clamped.

4.3 Boards, functions and devices

DECISION Combined boards are supported. Three words keep this precise:

TERM	MEANING	EXAMPLE
Board	One physical unit and its connection. It owns the transport, the <code>WireClient</code> and the worker thread.	ESP32 "Bench-5" over Wi-Fi at 192.168.10.41
Function	One instrument hosted by a board, with its own firmware model name and command namespace.	<code>photometer</code> and <code>thermometer</code> on Bench-5
Device	The app's object for one function, created by the device plugin whose <code>models</code> match that function. It has capabilities, parameters and a lease.	"Bench-5 · Photometer" (<code>LightSensor</code> , <code>LightSource</code>)

A single-function board is simply a board with one function. A Vernier sensor is a board without the wire protocol: its plugin talks to `godirect` directly, over USB or Bluetooth. Experiments only ever see devices, so a photometer behaves the same whether it sits alone on an Uno or next to a thermometer on an ESP32.

A device plugin is one `Device` subclass. Class attributes carry the plugin metadata; methods carry the behaviour. Everything below runs in its board's worker thread, so the code is plain, blocking Python.

```

class Device(ABC):
    # — plugin metadata —
    id: ClassVar[str]                                # "tsl2591_photometer"
    name: ClassVar[str]                              # "Photometer (TSL2591 + LED)"
    links: ClassVar[frozenset[str]]                  # {"serial", "tcp", "ble"} or {"godirect"} ...
    capabilities: ClassVar[frozenset[type]]
    connection: ClassVar[list[Parameter]] = []       # port, baud rate ...
    models: ClassVar[frozenset[str]] = frozenset()   # matched against the functions in the board's ID? reply
    simulated: ClassVar[type[Device] | None] = None
    gui: ClassVar[str | None] = None                 # "pkg.module:Panel", imported only by the GUI
    hardware: ClassVar[str | None] = "hardware/hardware.toml" # hardware sheet (§7.4)

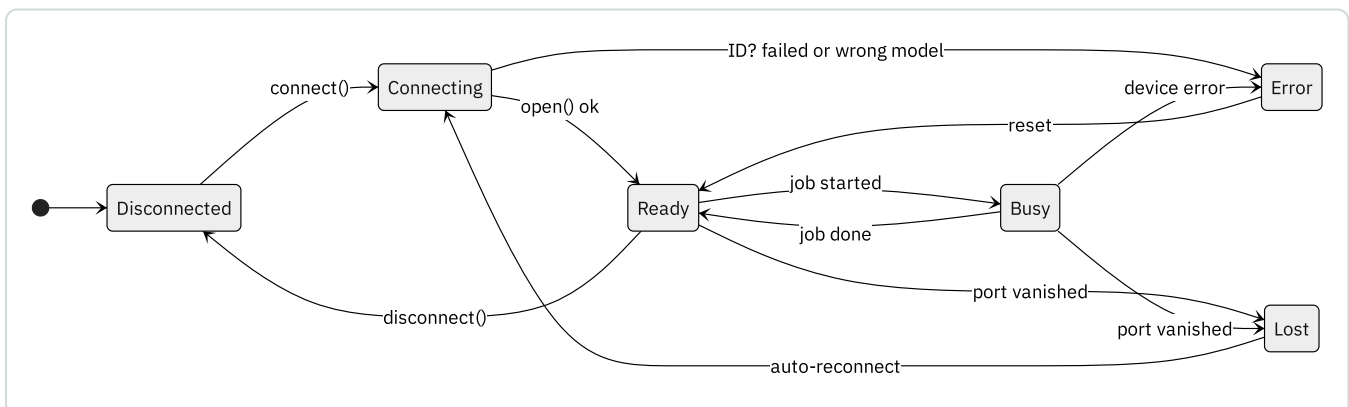
    # — lifecycle —
    def open(self) -> DeviceInfo: ...                # ID? check; read firmware version and current settings
    def close(self) -> None: ...
    def parameters(self) -> list[Parameter]: ...
    def set_parameter(self, key: str, value: Any) -> None: ...
    def commands(self) -> list[Command]: ...         # manual actions: Home, Calibrate, Clear buffer ...
    def poll(self) -> dict[str, float] | None: ...   # optional: live monitoring
    def operating_point(self) -> list[OperatingValue]: ... # optional: live figures and hints (§7.4)

class Tsl2591Photometer(Device, LightSensor, LightSource, Sensor):
    id = "tsl2591_photometer"
    name = _("Photometer (TSL2591 + LED)")
    links = frozenset({"serial", "tcp", "ble"}) # any link the board offers
    capabilities = frozenset({LightSensor, LightSource, Sensor})
    models = frozenset({"photometer"})          # ID? → "functions=photometer,..."
    simulated = SimulatedPhotometer

    def read_light(self, samples: int = 1) -> LightReading:
        r = self.wire.query("READ", samples)      # "photometer:READ 3" → Reply(bb=5691, ir=1899,
        sat=0)
        return LightReading(tsl2591_lux(r.int("bb"), r.int("ir"), self.int_time, self.gain),
                           r.int("bb"), r.int("ir"), saturated=r.bool("sat"))

```

Command s are identified by a key, and their label is only for display. In the current code two buttons share the label "Motor home" and one silently shadows the other; keying by id removes that failure.



4.4 Links and the Labint wire protocol

DECISION Every board speaks one text protocol, whether it is an Arduino, an ESP32, an STM32 or a Teensy, and whether it is connected over USB, Wi-Fi or Bluetooth. A new board then needs only its firmware and a device plugin; transports, discovery, stopping, error handling and the raw console work unchanged. The old ... executed protocol is not carried over; the existing boards are reflashed during M1 and M3.

Links

LINK	TRANSPORT	DISCOVERY	BOARDS	NOTES
USB	<code>SerialTransport</code> (pyserial)	List serial ports, then <code>ID?</code>	all	Records the USB serial number and VID:PID, so a board is recognised even when Windows changes its COM number.
Wi-Fi	<code>TcpTransport</code> ; the board is a TCP server on port 4550	mDNS service <code>_labint._tcp</code> (TXT: serial, name, functions) on the PC's own network; IP address given by the user for boards on another network	ESP32	A connection must start with <code>AUTH <token></code> . The lab PCs reach the boards directly over the school network.
Bluetooth	<code>BleTransport</code> (bleak), Nordic UART Service	BLE scan for devices advertising the service and a <code>LabInt-</code> name	ESP32	Lines split across notifications are reassembled. Enough for tens of readings per second.
Vernier	<code>godirect</code> , over USB or Bluetooth	godirect's own scan	Go Direct sensors	No wire protocol; the Vernier plugin talks to <code>godirect</code> directly. The link is a connection parameter (<code>usb</code> or <code>ble</code>). Battery and signal strength are shown.

- **The user chooses the link.** "Add device" lists what it found on every link (mockup 3). Each board record remembers its preferred link and a fallback order, for example Wi-Fi, then USB.
- **One board, one entry.** The same board seen on USB and Wi-Fi at once is listed once, identified by its serial number.
- **One host at a time.** A board serves the first host that sends it a command. Any other host gets `ERR 4 busy owner=tcp:192.168.10.23`, so two PCs can't drive the same stepper.
- **Wi-Fi setup happens over USB.** "Set up Wi-Fi..." on a USB-connected ESP32 sends `WIFI ssid="Lab 2" psk="..."` and reads the board's access token with `TOKEN?`. Credentials stay in the board's flash and are never shown again. `WIFI` and `TOKEN?` are only accepted over USB. If the board can't join the network, it starts its own access point named `LabInt-Bench-5` so it can still be reached for setup.
- **Boards on another network.** mDNS doesn't cross from one network to another, so a board on a different subnet or VLAN is added by IP address, which is saved with the board. So that the saved address stays valid, the Wi-Fi setup can give the board a static address (`WIFI ... ip=192.168.20.41/24 gw=192.168.20.1`), or the network can reserve one for it. `NET?` reports `ssid`, `ip` and `rssi` over any link. The Devices view shows it and offers "Update address..." when a saved address stops answering.
- **Bluetooth** uses the same `AUTH` token. On Linux it needs BlueZ; on Windows, the built-in Bluetooth LE support of Windows 10 and later.

Protocol rules (version 1)

- **Lines.** ASCII lines ending in `\n` (`\r\n` is accepted), at most 160 bytes. UART boards default to 115 200 baud; USB CDC boards ignore the rate. Values containing spaces are double-quoted.
- **Addressing.** Commands for a function carry its name as a prefix: `thermometer:READ?`. Board-level commands have no prefix. On a single-function board the prefix is optional, so simple sketches stay simple. Replies and events from a function include `fn=...`.
- **One command at a time.** The host waits for each reply before sending the next command. While one function is running an operation (after `BUSY`), the board still accepts commands for its *other* functions, plus `STOP`, `STATUS?` and `PING`.
- **Commands.** `VERB arg ...`. Verbs are upper case and queries end in `?`. Numbers always use `.` as the decimal separator, whatever the PC's locale.

- **Replies** start with `OK` , `BUSY` or `ERR` . Payloads are `key=value` pairs, so they describe themselves, their order doesn't matter, and the host ignores keys it doesn't know. This keeps old apps compatible with newer firmware.
- **Errors.** `ERR` code message . The standard codes are 1 unknown command, 2 bad argument, 3 out of range, 4 busy (or owned by another host), 5 not ready (for example "not homed"), 6 hardware fault, 7 not supported and 8 not authorised. Codes from 100 up are model-specific.
- **Events.** `EVT` name `key=value` ... lines can arrive at any time: `EVT DONE` , `EVT STOPPED` , `EVT LIMIT` , `EVT FAULT` , and `EVT BOOT` when the board restarts.
- **Debug lines.** Lines starting with `#` go to the device log and never break parsing, so firmware authors can print freely while they develop.
- **Timing.** A command must reply within 500 ms. Anything slower must answer `BUSY` at once and finish with an event. During a `BUSY` operation the host sends `STATUS?` every second; a board that stops answering is marked *Lost*.
- **Versioning.** `ID?` reports `proto=1` . New commands and keys are additive; only a breaking change raises the protocol number. The app supports the current and the previous version.

BOARD-LEVEL COMMAND	EXAMPLE REPLY	PURPOSE
<code>ID?</code>	<code>OK proto=1 board=esp32 fw=1.2.0 serial=3C71BF5A0E24 name=Bench-5 functions=photometer,thermometer links=usb,wifi,ble</code>	Discovery; one device per function, matched to plugins by function name.
<code>CMD5?</code>	<code>OK photometer=LED,LED?,CFG,CFG?,READ thermometer=READ?</code>	Conformance check; completion in the raw console.
<code>PING</code>	<code>OK</code>	Liveness.
<code>STATUS?</code>	<code>OK photometer=idle thermometer=idle</code>	Heartbeat during long operations.
<code>STOP , fn:STOP</code>	<code>OK</code> (then <code>EVT STOPPED ...</code> if something was running)	Stops every function, or just one; accepted at any time.
<code>RESET</code>	<code>OK</code>	Restores default settings.
<code>NAME Bench-5</code>	<code>OK</code>	Stores a friendly name on the board.
<code>AUTH <token></code>	<code>OK</code> / <code>ERR 8</code>	Required first on Wi-Fi and Bluetooth.
<code>WIFI ... , TOKEN?</code>	<code>OK</code> , <code>OK token=K7Q2M9XD</code>	Wi-Fi setup (DHCP or static IP) and token reading; USB only.
<code>fn:HW?</code> (optional)	<code>OK sensor=ts12591 leds=rgbo-5mm</code>	The parts a function is built with, so the plugin shows the right hardware sheet and uses the right formulas (§7.4).
<code>NET?</code>	<code>OK ssid="Lab 2" ip=192.168.20.41 rssi=-58</code>	Network status, shown in the Devices view.

```

# combined board: ESP32 "Bench-5" over Wi-Fi
→ AUTH K7Q2M9XD          ← OK
→ ID?                    ← OK proto=1 board=esp32 fw=1.2.0 ... functions=photometer,thermometer
→ photometer:LED red 1850 ← OK
→ photometer:READ 3      ← OK fn=photometer bb=5691 ir=1899 sat=0
→ thermometer:READ?     ← OK fn=thermometer t=21.37
→ photometer:LED green 5000 ← ERR 3 power out of range 780..900

# single-function board: Teensy 4.1 "Rail-1" over USB (prefix optional)
→ LIMITS?                ← OK fn=linear-actuator travel=380.980 step=0.00125 homed=1
→ MOVE 42.5              ← BUSY fn=linear-actuator
                        ← EVT DONE fn=linear-actuator cmd=MOVE pos=42.500
→ MOVE 300               ← BUSY fn=linear-actuator
→ STOP                   ← OK
                        ← EVT STOPPED fn=linear-actuator cmd=MOVE pos=137.410
                        ← # driver temp 41C          # debug line → device log

```

PC side: WireClient

The board owns one `WireClient`, and each device gets a `FunctionChannel` from it that adds the prefix, so plugin code just writes `self.wire.query("READ", 3)`.

- `query(verb, *args, timeout=0.5)` returns a `Reply` with typed accessors (`.int("bb")`, `.float()`, `.bool()`). A missing key raises `BadReply`, and `ERR` becomes `DeviceError(code, message)`.
- `start(verb, *args)` expects `BUSY` and returns an `Operation`: a future completed by the matching `DONE`, `STOPPED` or `LIMIT` event (§4.5).
- Unsolicited events are dispatched to the device named in `fn=`: `LIMIT` and `FAULT` change its state and show in the GUI, and `BOOT` re-initialises the whole board.

Arduinos that reset when the USB port opens announce themselves with `EVT BOOT`. The client waits for that event (at most 3 s) instead of sleeping for a fixed time.

Firmware side: the LabInt C++ library

The library is written against the Arduino core API, so the same code builds for AVR (Uno, Mega, Nano), ESP32 (Arduino-ESP32), STM32 (STM32duino) and Teensy (Teensyduino). A small platform layer covers what differs between them:

PLATFORM	UNIQUE SERIAL	SETTINGS STORAGE	LINKS
AVR	random, generated on first boot and kept in EEPROM	EEPROM	USB
ESP32	eFuse MAC	NVS (<code>Preferences</code>)	USB, Wi-Fi, Bluetooth
STM32	96-bit device UID	EEPROM emulation in flash	USB
Teensy	hardware serial number	EEPROM	USB

It uses fixed buffers and no heap, and keeps strings in flash, so it fits an Uno with 2 KB of RAM. Wi-Fi and Bluetooth are compiled in only when enabled with build flags. A sketch declares its functions and their commands:

```
// firmware/sketches/combo-esp32/src/main.cpp (abridged)
#include <LabInt.h>
LabInt board("1.2.0");
LabInt::Function& photo = board.function("photometer");
LabInt::Function& thermo = board.function("thermometer");

void cmdRead(LabInt::Request& rq) {
    int n = rq.intArg(0, /*default*/ 1, /*min*/ 1, /*max*/ 15);    // sends ERR 3 by itself if out of range
    if (!rq) return;
    Reading r = sensor.average(n);
    rq.ok().kv("bb", r.broadband).kv("ir", r.infrared).kv("sat", r.saturated).send();
}

void setup() {
    photo.on("READ", cmdRead); photo.on("LED", cmdLed); thermo.on("READ?", cmdTemp);
    board.addLink(LabInt::serialLink(Serial, 115200));
    #if LABINT_WIFI
        board.addLink(LabInt::wifiLink(4550));    // credentials from NVS, set over USB
    #endif
    #if LABINT_BLE
        board.addLink(LabInt::bleLink());    // Nordic UART Service, advertised as "LabInt-Bench-5"
    #endif
    board.begin();
}
void loop() { board.poll(); }    // never delay(): long work runs as a state machine, so STOP is always heard
```

Sketches are built with **PlatformIO**. Each has a `platformio.ini` with one environment per supported board (uno, esp32dev, nucleo_f446re, teensy41 ...), and CI compiles all of them. The parser is unit-tested on the PC with PlatformIO's `native` environment. The library is also published as a zip, so it can be installed in the Arduino IDE.

`labdaemon firmware check <port|host|ble-name>` checks any board, over any link, against the spec. It runs every board-level command, measures reply latency, checks that unknown verbs give `ERR 1` and that `STOP` is accepted while idle, then runs each matching plugin's own firmware tests. In the Python test suite, the `WireSimulator` implements the same spec, so the spec is executable.

4.5 Threads: one worker per connection

Your "one worker thread per device" rule, made precise for combined boards: the worker belongs to the *connection*, because that is what cannot be shared. A single-function board or a Vernier sensor gets exactly one worker. The photometer and thermometer of a combined ESP32 share their board's worker, and their commands are interleaved on it.

THREAD	COUNT	OWNS	NEVER DOES
GUI (Qt main)	1	widgets, plots, input	blocking I/O, <code>sleep</code>
Board worker	1 per connection	the transport, the <code>WireClient</code> and every device of that board	touching widgets; waiting for a long operation to finish
Experiment runner	1 per open experiment	the experiment's actions and runs, one at a time	touching transports directly

```

class BoardWorker:
    """The only thread that ever touches its board's transport and devices."""
    def submit(self, fn, /, *args, urgent=False, **kw) -> Future: ... # urgent jobs (STOP) jump the queue
    def call(self, fn, /, *args, timeout=None, **kw): ... # submit + wait (runner thread,
Jupyter)
    def proxy(self, device) -> DeviceProxy: ... # same capability methods, each one a
call()
    def start_polling(self, device, interval_s: float): ... # live values → sample event
    def shutdown(self): ...

    def _loop(self):
        while not self._stopping:
            self._read_incoming() # replies; EVT → complete Operations, notify devices; "#" → log
            job = self._next_job(timeout=self._until_next_deadline())
            if job: self._execute(job) # result or exception → job.future; updates device state
            self._run_due_polls()
            self._send_heartbeats() # STATUS? while any Operation is pending

```

- Calls to one board are serialized, so two callers can never interleave bytes on the same link.
- Polling runs between queued jobs, so a live reading never delays a "Set power" click.
- **Long operations never block the worker.** `start()` sends the command, receives `BUSY` and returns an `Operation` at once. The worker keeps serving the board's other functions, polling and `STOP` while the stepper moves, and completes the `Operation` when its event arrives.

Experiment code receives **proxies**. `self.sensor.read_light(3)` looks like a direct call but runs on the board's worker. When a device method returns an `Operation`, the proxy waits for it on the *caller's* thread. The same experiment code therefore runs unchanged in the GUI (on the runner thread), in Jupyter and in tests.

Stopping a move

Stop is immediate. If the run is cancelled (Stop button, Esc) while a proxy is waiting on an `Operation`, the proxy submits the device's `stop()` as an *urgent* job. That job jumps the queue, and the worker is never stuck in a wait, so `STOP` reaches the board within milliseconds.

```

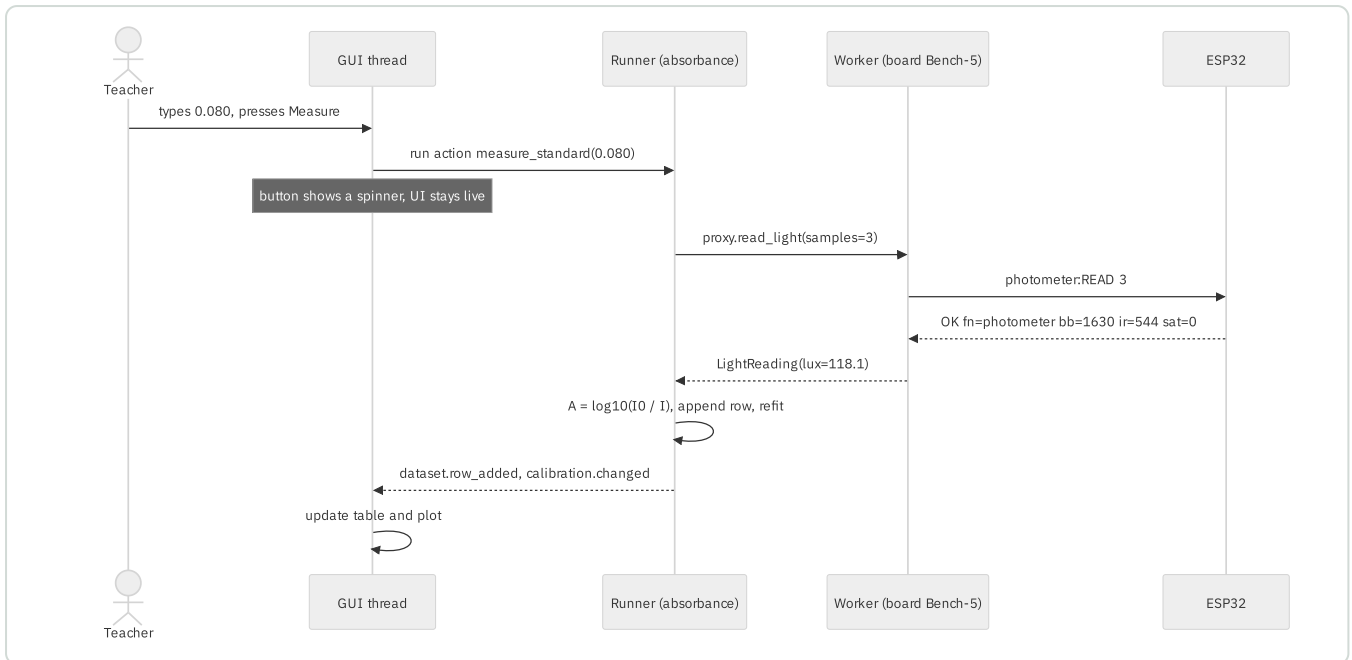
class StepperActuator(Device, LinearActuator):
    def move_to(self, mm: float) -> Operation:
        return self.wire.start("MOVE", f"{mm:.3f}") # ← BUSY; returns at once

    def stop(self) -> None:
        self.wire.query("STOP")

# inside a field-scan run, on the experiment's runner thread:
self.actuator.move_to(z) # proxy waits for EVT DONE; on cancel it sends STOP (urgent)
                        # EVT STOPPED → Interrupted(position), EVT LIMIT → LimitReached(side, position)

```

`Interrupted` ends a field scan cleanly: the points already measured stay in the dataset, and the session records that the run was stopped and where.



4.6 Experiments: actions and runs

An experiment is a Qt-free controller class that holds state and exposes two kinds of operations:

- **@action** : a short operation that returns a result, for example "measure blank" or "measure this standard".
- **@run** : a long acquisition that records into a dataset until it finishes or is stopped, for example kinetics or a field scan.

The wizard and the expert panel are two views that call the same operations. Declaring them also lets the GUI build a *generic expert panel*: a plugin with no Qt code at all still gets a parameter form, action buttons, run controls, a plot and a table.

```

class Absorbance(Experiment):
    id = "absorbance"
    name = _("Absorbance and concentration")
    category = "chemistry"
    requires = {
        "sensor": Requirement(LightSensor, _("Light sensor")),
        "source": Requirement(LightSource, _("Light source")),
    }
    parameters = [
        Parameter("samples", _("Readings per measurement"), int, 3, minimum=1, maximum=15),
        Parameter("path_length", _("Cuvette path length"), float, 1.0, unit="cm", advanced=True),
    ]
    gui = "labdaemon.experiments.absorbance.gui:AbsorbanceGui"  # optional

    @action(_("Measure blank"))
    def measure_blank(self) -> LightReading: ...

    @action(_("Measure standard"))
    def measure_standard(self, concentration: float) -> None: ...

    @action(_("Measure unknown"))
    def measure_unknown(self) -> Concentration: ...  # value ± uncertainty, mol/L

class ReactionKinetics(Experiment):
    ...
    @run(_("Record"))
    def record(self, ctx: RunContext, interval_s: float = 2.0, duration_s: float | None = None):
        ds = self.session.dataset("kinetics", [T, LUX, ABSORBANCE])
        while not ctx.cancelled and (duration_s is None or ctx.elapsed < duration_s):
            r = self.sensor.read_light(self.params["samples"])
            ds.append(t=ctx.elapsed, lux=r.lux, A=absorbance(self.blank.lux, r.lux))
            ctx.sleep_until_next(interval_s)  # interruptible, no drift

```

`RunContext` provides `cancelled`, `elapsed` (monotonic seconds since the run started), interruptible `sleep` / `sleep_until_next`, `progress(fraction, text)` and `warn(message)`. Stopping a run cancels the context; any device operation the run is waiting on receives an urgent `STOP` (§4.5).

4.7 Data, sessions and calibrations

- `Channel` (key, label, unit) and `Dataset`: columnar, numpy-backed, append-only while recording, with a thread-safe `append` and a `row_added` event. Exports: `to_csv`, `to_numpy` and `to_pandas` (pandas is optional). The time column `t` is *seconds since the run started*; the absolute start time is stored as ISO 8601 with a time zone.
- `Session` holds everything one lab produces: the experiment id and version, the mode used, parameter values, a device snapshot (board, function, link, firmware and settings at start), the calibrations used, datasets, notes and a log. **DECISION** One session is one `.labint` file: a zip containing `session.json`, `datasets/*.csv` and `log.txt`, so any unzip tool can read it. During runs, sessions autosave to a recovery folder. They are saved to `Documents/LabDaemon` by default, or to a class folder set by policy (§11).
- File names such as `2026-09-28_1432_absorbance_copper-sulfate.labint` contain no `:`, so they are valid on Windows.
- CSV export presets: *International* (, separator, . decimal) and *Excel Italia* (; separator, , decimal). XLSX export is available with an optional extra.
- `Calibration` is a first-class object kept in a calibration library and reusable across sessions. It stores the conditions it is valid for. Applying a calibration sets those conditions atomically (colour first, then power), which fixes today's "loading resets the LED power" bug, and the app warns when current conditions differ.

```
{ "kind": "absorbance.linear", "label": "CuSO4 · red LED", "created": "2026-09-24T10:12:00+02:00",
  "device": { "plugin": "ts12591_photometer", "serial": "A1B2C3" },
  "conditions": { "led_color": "red", "led_power": 1850, "integration_time_ms": 100, "gain": 25,
    "path_length_cm": 1.0 },
  "points": [[0.0, 0.000], [0.02, 0.138], [0.04, 0.268], [0.06, 0.409]],
  "fit": { "slope": 6.785, "slope_err": 0.071, "intercept": 0.0002, "r2": 0.9998 } }
```

Analysis

DECISION `core.analysis` uses `scipy` and replaces `scikit-learn`. Every fit returns parameters with uncertainties, R^2 , residuals and a readable formula, which are more useful in class than a bare slope.

```
@dataclass(frozen=True)
class FitResult:
    model: Model
    params: dict[str, Measured]          # Measured(value, stderr, unit): prints "6.785 ± 0.071 L/mol"
    r2: float
    residuals: np.ndarray
    def predict(self, x): ...
    def formula(self) -> str: ...        # "A = 6.785·c + 0.000"

def fit(model: Model, x, y, sigma=None, p0=None) -> FitResult:
    """Linear models: closed form. Others: scipy.optimize.curve_fit; errors from sqrt(diag(cov))."""
```

BUILT-IN MODEL	FORMULA	USED BY
linear, proportional	$y = m \cdot x + q$, $y = m \cdot x$	absorbance calibration (Beer–Lambert), generic
first-order kinetics	$A(t) = A_{\infty} + A_0 \cdot e^{(-k \cdot t)}$	kinetics; reports k and $t_{1/2}$
second-order kinetics	$1/[X] = 1/[X]_0 + k \cdot t$	kinetics (with a calibration, on concentration)
finite solenoid (on axis)	$B(z) = \frac{1}{2} \mu_0 n I \cdot [\cos \alpha_1 - \cos \alpha_2]$, fitting centre, length, radius and $n \cdot I$	field profile
dipole far field	$B(r) = C / (r - r_0)^3 + B_0$	field profile (permanent magnets)

Experiments choose which models to offer, and plugins can register their own (`Model(name, func, params, p0_guess)`). The blank ($c = 0$) is part of the calibration fit, and fitting through the origin is an option.

4.8 Events, errors and logging

- `core.events.Signal` (`connect`, `disconnect`, `emit`) runs callbacks in the emitting thread. The GUI bridge re-emits them as queued Qt signals so widgets update on the GUI thread; in Jupyter they are plain callbacks.
- Exceptions share one hierarchy. `LabError` is the base, with `TransportError` (`PortNotFound`, `PortBusy`, `PermissionDenied`), `ProtocolError` (`ProtocolTimeout`, `BadReply`, `UnsupportedProtocol`), `DeviceError` (carries the `ERR` code; subclasses `LimitReached` and `Interrupted`), `PluginError` and `ExperimentError` (for example `NotReady`: "Measure the blank first"). Each error carries an actionable user message and a technical detail. For example: "Permission denied on /dev/ttyUSB0. Add your user to the `dialout` group, then log out and back in."
- Logging stays on `loguru`, with three sinks: a rotating file in the user log folder, the GUI log dock, and the session log, which is saved with the data.

5 Plugin system

DECISION Built-in plugins are declared as entry points. Development and unpackaged plugins live in plugin folders and describe themselves in a `plugin.toml` with *the same table shape*, so a single loader handles both.

```
# pyproject.toml: built-in plugins
[project.entry-points."labdaemon.transports"]
serial =
"labdaemon.transports.serial:SerialTransport"
simulated =
"labdaemon.transports.simulated:SimulatedTransport"

[project.entry-points."labdaemon.devices"]
tsl2591_photometer =
"labdaemon.devices.tsl2591:Tsl2591Photometer"
stepper_actuator =
"labdaemon.devices.stepper:StepperActuator"
vernier_godirect =
"labdaemon.devices.vernier:GoDirectDevice"

[project.entry-points."labdaemon.experiments"]
absorbance =
"labdaemon.experiments.absorbance:Absorbance"
kinetics =
"labdaemon.experiments.kinetics:ReactionKinetics"
field_profile =
"labdaemon.experiments.field_profile:FieldProfile"
data_logger =
"labdaemon.experiments.data_logger:DataLogger"
```

```
# plugins/spectrometer/plugin.toml: folder plugin
[plugin]
id = "spectrometer"
name = "Mini spectrometer"
version = "0.1.0"
api = 1
authors = ["Matteo Mosangini"]
needs = ["numpy"] # checked at load time

[devices]
ccd_spectrometer = "device:CcdSpectrometer"

[experiments]
emission_spectrum = "experiment:EmissionSpectrum"

# layout: plugins/spectrometer/{plugin.toml,
device.py, experiment.py}
```

Where folder plugins are searched

In this order: the `--plugin-dir` command-line option, the `LABDAEMON_PLUGIN_PATH` environment variable, the user plugin folder (Linux: `~/.local/share/labdaemon/plugins`, Windows: `%LOCALAPPDATA%\labdaemon\plugins`), then the machine-wide folder for central installs (Linux: `/usr/local/share/labdaemon/plugins`, Windows: `%PROGRAMDATA%\LabDaemon\plugins`). The machine policy can disable the user folders, so students' accounts only get the plugins IT has approved (§11). Each plugin is imported under a private namespace (`labdaemon_ext.<id>`) so its module names cannot clash with anything else.

Loading pipeline

1. Collect entry points and manifests. Nothing is imported yet, so listing is fast and safe.
2. Check `api` against `core.API_VERSION`. A mismatched plugin is listed as incompatible and not loaded.
3. Import the plugin. A failure marks that plugin *Error* with its traceback in the Plugins view; the app starts anyway.
4. Validate: metadata present, capabilities declared and actually implemented, requirements that point to known capabilities, unique ids.
5. Register. A folder plugin with the same id as a built-in one overrides it *only* in developer mode, and a banner says so. This lets you test a new version of a built-in plugin without touching the installed app.

Developer tooling

- **Reload development plugins** (Plugins view) reimports folder plugins whose devices and experiments are not in use.
- `labdaemon plugin new device|experiment <name>` creates a working skeleton in the plugin folder.

- `labdaemon plugin check <path>` validates the manifest, imports the plugin, checks capability conformance, and runs every action against simulated devices. It is useful once other authors join.

6 Guided and expert modes

Both modes are views of the same `Experiment` object and session. Switching modes mid-lab keeps every measurement.

SITUATION	OPENS IN	NOTES
Student mode on	Guided, always	This is the student lock (§6.1): only experiments the teacher has enabled, advanced options hidden, and teacher tools locked behind a PIN.
Teacher, first time with this experiment	Guided	A "Skip to expert mode" link is always visible. The last step offers "Next time open in expert mode", checked by default.
Teacher, afterwards	Last mode used (expert by default)	The Guided ⇄ Expert switch in the toolbar works at any time.

6.1 Student lock

DECISION Student mode is a lock that a teacher opens with a PIN. It protects the setup from accidental changes during a lesson. It is not a security boundary against a determined student with access to the computer.

AREA	LOCKED (STUDENT)	UNLOCKED (TEACHER)
Experiments	Guided mode only. Only experiments marked "available to students" are listed.	All experiments, both modes, and the per-experiment "available to students" switch.
Parameters	Non-advanced parameters only, within limits the teacher can narrow (for example LED power 1500–2200).	Everything.
Devices	Connect and disconnect known boards on their saved link. The Hardware tab (§7.4) is visible, read-only.	Also discovery on every link, Wi-Fi setup, renaming, the raw console and the firmware check.
Calibrations	Use them; save new ones as their own.	Also mark calibrations as shared, and edit or delete them.
Firmware	Hidden.	Update, compile, upload and edit (§7.3).
Plugins, Settings	Hidden.	Visible.
Saving	A group or student name is asked when an experiment starts, and prefixes the file name. Sessions go to the class folder, or Documents if none is set.	Anywhere.

- **Unlocking.** Teacher unlock... (Ctrl+Shift+T) asks for the PIN. Open experiments stay open and their extra options appear in place. The lock returns after 10 minutes without input (configurable), at restart, or with *Lock now*.
- **The PIN** has 4–8 digits and is stored as a salted `script` hash. After five wrong attempts, the next try is delayed by a minute.
- **Where the lock lives.** In a central install, the PIN hash and the setting `student_mode = "forced" | "default" | "off"` live in the machine policy file, which only administrators can write. A student then can't remove the lock by deleting their own settings. In a per-user install they live in the user's settings.
- **A forgotten PIN** is reset by an administrator editing the policy file, or, in a per-user install, by deleting one settings file. The user guide documents both.

Declarative wizard steps

Wizard steps are data defined in `core/wizard.py`, and the GUI renders them. Plugin authors describe a guided flow without writing Qt code, and half-finished wizards resume at the right step when a session is reopened.

STEP	SHOWS	COMPLETE WHEN
<code>DevicesStep</code>	each requirement with its matching devices; auto-selects when only one fits; "Connect..."	all required roles are filled
<code>ParametersStep</code>	chosen parameters (never advanced ones) with an optional live reading	values valid and the optional check passes (for example "not saturated")
<code>InstructionStep</code>	text and an illustration: "Fill a cuvette with distilled water..."	the user confirms
<code>ActionStep</code>	one button that runs an <code>@action</code> , its result, and a validator	the action succeeded and the validator accepts the result
<code>TableStep</code>	an editable table: type a value and the row action runs; live plot	<code>min_rows</code> reached
<code>RunStep</code>	start and stop for a <code>@run</code> , with a live plot	the run finished or was stopped
<code>ResultStep</code>	results, fits, save and export	–
<code>CustomStep</code>	a Qt widget from the plugin's <code>gui.py</code>	the widget reports completion

```
def wizard(self) -> list[Step]:
    return [
        DevicesStep(),
        ParametersStep(["led_color", "led_power"], live="sensor",
                       check=lambda r: not r.saturated or _("Sensor saturated: lower the LED power")),
        InstructionStep(_("Fill a cuvette with distilled water and insert it."), image="blank.svg"),
        ActionStep(self.measure_blank, shows=["lux"]),
        TableStep("standards", input="c", row_action=self.measure_standard, plot="calibration", min_rows=3),
        ActionStep(self.measure_unknown, label=_("Unknown sample")),
        ResultStep(self.summary),
    ]
```

7 GUI architecture

- **Main window:** a navigation rail (Experiments, Devices, Data, Plugins, Settings), a workspace, and a status bar with one chip per device showing its state, port and which experiment holds it.
- **Experiments open as tabs.** Several can be open and recording at once, each holding *leases* on the devices it controls (§7.2).
- **Expert workspace:** a nested `QMainWindow` per experiment with dock widgets for setup, plot(s), data table, analysis and log. The layout is saved per experiment, with a "Reset layout" command.
- **Shared widgets** in `gui/widgets` :
 - `ParameterForm` is generated from parameters.
 - `LivePlot` is pyqtgraph with a ring buffer. It redraws at up to 30 Hz from a timer instead of once per sample. Its auto-range adds 5 % padding, so no point is clipped. It has series toggles, a crosshair readout, and PNG/SVG export.
 - `DatasetTable` is a `QAbstractTableModel` over a dataset; copying gives TSV, which pastes into Excel or LibreOffice.
 - `LogView`, `DeviceChip` and `StepRail`.

- **Bridge:** `QtBridge` turns core signals and futures into queued Qt signals, and `watch(future, on_ok, on_error)` handles asynchronous calls from widgets.
- **Look:** the Fusion style with the app's own palette, following the system light/dark setting (Qt ≥ 6.5), with High-DPI support. **Presentation mode** (F11) enlarges fonts and plot lines for the classroom projector.
- **Translations:** English by default, Italian available. See §7.1.
- **Keyboard:** Space measures in the current step, Ctrl+R starts a run, Esc stops a run or a moving actuator. Device state is shown in text as well as colour, and the plot series palette is safe for colour-blind users.

7.1 Languages

DECISION The interface is English by default, with Italian available. All source strings are written in English.

WHERE THE TEXT LIVES	MECHANISM	FILES
GUI (<code>gui/</code> , plugin <code>gui.py</code>)	Qt <code>self.tr()</code> ; <code>pyside6-lupdate</code> extracts, Qt Linguist translates, <code>pyside6-lrelease</code> builds	<code>gui/i18n/labdaemon_it.ts</code> → <code>.qm</code>
Core and plugin logic: parameter labels, wizard texts, error messages	<code>gettext _()</code> and lazy <code>N_()</code> for class-level metadata; Babel extracts and compiles	<code>locale/it/LC_MESSAGES/labdaemon.po</code> ; each plugin has its own <code>locale/</code> and domain
Wizard illustrations	No text inside images; labels are drawn as overlays from translated strings	<code>experiments/*/assets/*.svg</code>
Documentation	A separate page per language	<code>docs/en/</code> , <code>docs/it/</code>

- The language is chosen in Settings; English is the default even on an Italian system. The change applies after a restart, which avoids re-translating every open widget in v1.
- Numbers on screen follow the chosen language (`0.080` in English, `0,080` in Italian), and input fields accept both separators. Files and the wire protocol always use `.` . Choosing Italian also suggests the *Excel Italia* CSV preset.
- Units and symbols (mol/L, lx, mT, ε) are not translated.
- Plugins declare their languages in `plugin.toml` (`languages = ["en", "it"]`). A missing translation falls back to English, never to an empty label.
- CI fails if a built-in string has no Italian translation, and `plugin check` reports untranslated strings for plugins that declare Italian.

7.2 Several experiments at once: leases

DECISION Experiments open in tabs, and several can record at the same time, for example kinetics on Bench-5's photometer while a field scan runs on Rail-1.

- **A lease is the exclusive right to control one device** (one function of a board). An experiment takes leases when its devices are assigned, and releases them when its tab closes or a device is reassigned.
- **Watching needs no lease.** The Devices view and other tabs can show live readings from a leased device. One polling loop feeds every viewer.
- **Conflicts are explicit.** Asking for a leased device shows who holds it. If the holder is idle, *Take over* moves the lease and the other tab shows "device released". If the holder is recording, the request is refused with the way out: "Stop the run in 'Kinetics' first."
- **Combined boards.** Leases are per function, so the photometer and the thermometer of one ESP32 can serve two different experiments. Their commands interleave on the board's worker (§4.5).
- **Stopping.** Esc stops only the current tab's operations, using `fn:STOP` for its own devices. *Stop everything* (toolbar, Ctrl+Shift+Esc) sends `STOP` to every board.

- **Tab**s show a recording dot, and status-bar chips name the lease holder. Hidden tabs keep recording and buffering, and redraw their plots when shown.

7.3 Firmware view: update, compile, upload and edit

DECISION Teachers can manage board firmware without leaving LabDaemon. Compiling uses `arduino-cli`, preferably the copy bundled with an installed Arduino IDE 2, and editing uses the **Monaco** editor (the VS Code editor) embedded with `qtmonaco`. The feature has three levels, each usable on its own:

LEVEL	WHAT THE TEACHER CAN DO	NEEDS ON THE PC
1. Update	"Update firmware..." in the Devices view flashes the reference firmware of the current release onto a board. No editing or compiling.	Nothing extra for AVR and ESP32: each release ships precompiled images, plus <code>avrdude</code> and <code>esptool</code> . Teensy and STM32 use the upload tools of an installed board core.
2. Compile and upload	Open a sketch, compile and upload it to a connected board. Compiler errors are listed and clickable.	<code>arduino-cli</code> and the core of the board being compiled for.
3. Edit	Edit sketches in Monaco: C++ colouring, folding, find and replace, multiple cursors, and autocompletion, hover help and live errors from the Arduino language server.	As level 2, plus <code>arduino-language-server</code> and <code>clangd</code> (both shipped with Arduino IDE 2). The embedded web engine is included in LabDaemon's installers.

Core: `core/firmware.py` (no Qt)

```
@dataclass(frozen=True)
class Toolchain:
    cli: Path # arduino-cli
    language_server: Path | None # arduino-language-server (editor features)
    clangd: Path | None
    source: str # "Arduino IDE 2" | "PATH" | "settings"

    @classmethod
    def discover(cls, settings) -> Toolchain | None:
        """1. path from policy/settings 2. Arduino IDE 2 install 3. arduino-cli on PATH."""
        # IDE 2 keeps its tools in <install>/resources/app/lib/backend/resources/, e.g.
        # /opt/arduino-ide/... or %LOCALAPPDATA%\Programs\Arduino IDE\...

    def compile(sketch: Path, fqbn: str, *, on_output) -> BuildResult: ...
        # arduino-cli compile --fqbn ... --library <LabInt shipped with this release>
        # -> diagnostics (file, line, column, severity, message), flash and RAM use

    def upload(build: BuildResult, board: Board, *, on_output) -> FirmwareInfo: ...
```

The same functions power `labdaemon firmware build` and `labdaemon firmware upload` on the command line. The LabInt library is always passed from LabDaemon's own installation, so a sketch compiles against the library version the app expects.

- **Board type.** The compiler's board identifier (FQBN) comes from the board's `ID?` reply through a built-in table: `uno` → `arduino:avr:uno`, `esp32` → `esp32:esp32:esp32`, `teensy41` → `teensy:avr:teensy41`, and so on. The teacher can override it, and the choice is saved with the board.
- **Upload sequence.**
 1. Check leases. If an experiment is recording with the board, the upload is refused with the way out ("Stop the run in 'Kinetics' first"). Idle leases are paused.
 2. Close the board's connection; its worker stops.
 3. Upload over USB.
 4. Reconnect, wait for `EVT_BOOT`, and check that `ID?` reports the new firmware version.
 5. Run the firmware check, show the result, and resume the paused leases.

Boards connected only by Wi-Fi or Bluetooth are asked to be plugged in by USB; over-the-air updates for ESP32 can come later.

- **Board support.** "Install board support..." runs `arduino-cli core install`, adding the extra package URLs for ESP32, STM32 and Teensy, and shows progress and disk use (the ESP32 core takes a few hundred MB). In a central install, the policy can point `arduino-cli` at a shared data folder so each account doesn't download its own copy.
- **Sketches.** Reference sketches ship with each release and are read-only. "Save as copy" puts an editable copy in `Documents/LabDaemon/firmware`. "New sketch from template..." asks for the board and its functions (for example photometer + thermometer), then generates both the sketch, with those functions declared, and a matching device-plugin skeleton in the plugin folder.

Editor: Monaco through `qtmonaco`

- **Loaded on demand.** The web engine and Monaco start the first time the Firmware view opens, so the rest of the app never pays their memory cost.
- **Language server.** For the open sketch, LabDaemon starts `arduino-language-server` (which runs `clangd`) with the sketch's board identifier and `arduino-cli` settings. A small local WebSocket bridge relays its messages to Monaco. This is how Arduino IDE 2 itself edits sketches, so teachers get the same completion and error checking (early check `C5`).
- **Compiler errors** appear both in the Problems list and as markers in the editor. Ctrl+B compiles and Ctrl+U uploads.
- **"Open in Arduino IDE"** is always available, for the IDE's serial plotter or library manager, for example.
- **Fallback.** If the language server is missing, the editor keeps colouring and compiler markers without completion. If the web engine can't start, a plain text editor with the same Compile and Upload buttons is used.
- **Student lock.** The Firmware view is for teachers only (§6.1).

7.4 Hardware sheets: what the sensor is and what the numbers mean

DECISION Every device plugin describes the hardware behind it: parts, key datasheet figures, charts, and how raw values become the measured quantity. The Devices view shows this on a **Hardware** tab (mockup 9), next to a live *operating point*. That operating point turns the current settings and readings into terms a teacher or student can act on: how close the sensor is to saturating, how fine the resolution is, and what to change.

What a plugin ships

```
devices/tsl2591/hardware/
├ hardware.toml           # parts, specs, links, charts, which formula is used
├ notes.en.md, notes.it.md # explanations: how lux is computed, why the IR channel matters ...
├ charts/tsl2591_spectral.csv # wavelength_nm, ch0, ch1 (normalised, digitised from the datasheet)
└ datasheets/             # PDF copies, only when the manufacturer allows redistribution
```

```

[[component]]
id = "tsl2591"
role = "Light sensor"
part = "TSL2591"
manufacturer = "ams-OSRAM"
interface = "I2C, address 0x29"
datasheet_url = "https://ams-osram.com/.../tsl2591"
datasheet_file = "datasheets/TSL2591.pdf" # optional

[component.specs]
channels = [{ key = "ch0", name = "Full spectrum (visible + IR)" },
            { key = "ch1", name = "Infrared" }]
adc_bits = 16
integration_ms = [100, 200, 300, 400, 500, 600]
gain = { low = 1, medium = 25, high = 428, max = 9876 } # nominal factors
full_scale_counts = { "100" = 36863, default = 65535 }
lux_range = [188e-6, 88000]
lux_formula = "(ch0 - ch1) * (1 - ch1/ch0) / (t * g / 408)"

[component.charts]
spectral_response = { file = "charts/tsl2591_spectral.csv", x = "wavelength_nm" }

[[component]]
id = "led-set"
role = "Light source"
part = "4 × 5 mm LEDs"
[component.specs]
peak_wavelength_nm = { red = 625, orange = 605, green = 520, blue = 470 }

```

- **Specs are data, not prose.** Because the figures are structured, the plugin's own code uses them: the saturation check reads `full_scale_counts` instead of a hard-coded constant. The GUI and the Jupyter facade read the same file, so what the teacher sees and what the code does are always the same.
- **Operating point.** A device can implement `operating_point()`, which returns a few labelled values computed from its current settings and last reading. For the photometer:
 - each channel as a percentage of full scale;
 - the resolution in lux per count;
 - the light level at which it would saturate;
 - hints such as "switching to gain 428× would saturate at about 160 lx".

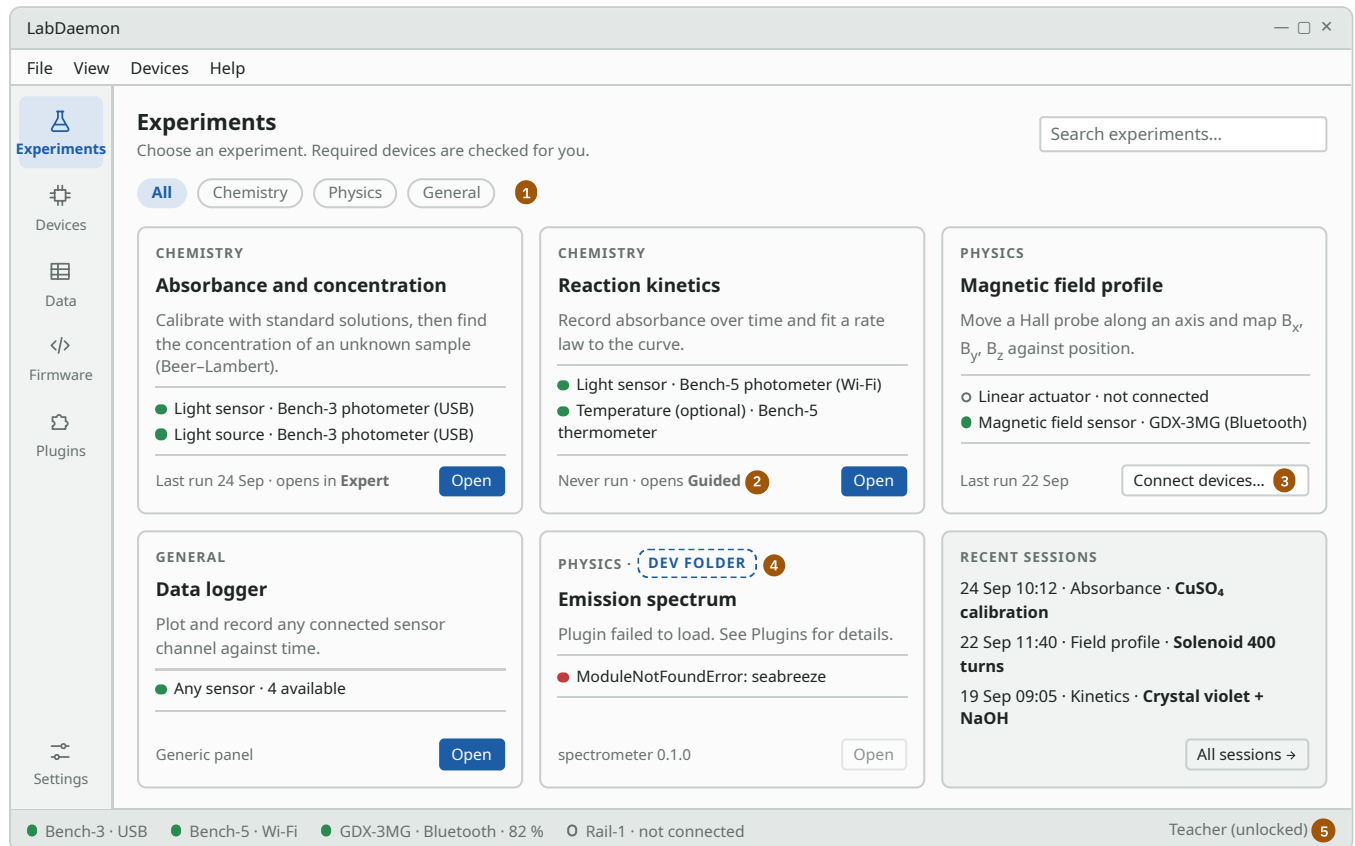
It runs on the board's worker thread like any other device call, and is refreshed with each live reading.

- **The board confirms its parts.** An optional function command, `HW?` (for example `OK sensor=tsl2591 leds=rgbo-5mm`), lets one plugin support variants, such as an older photometer built with a TSL2561. The plugin then shows the right sheet *and* uses the right lux formula. Without `HW?`, the plugin's default parts are assumed and marked "not confirmed by the board".
- **Datasheets.** The link to the manufacturer's page is always shown. A PDF copy ships only where redistribution is allowed. A teacher can also attach their own copy, stored in the data folder or the shared folder set by the policy, so it works offline in class. PDFs open in a built-in viewer (Qt PDF, part of `PySide6-Addons`, which the Firmware view already needs), or in the system viewer.
- **Notes** are short Markdown explanations per language, rendered on the tab. Wizard steps can link to a section, for example a ParametersStep's "What do gain and integration time do?".
- **Vernier sensors.** Their sheets link to Vernier's product pages. Ranges and resolution come from what `godirect` reports about each sensor channel.
- **Students see it too.** The Hardware tab is read-only and stays visible when the student lock is on, because it is teaching material.

- **From Jupyter:** `photo.hardware.specs["full_scale_counts"]`,
`photo.hardware.chart("spectral_response")` (a table) and `photo.operating_point()`.

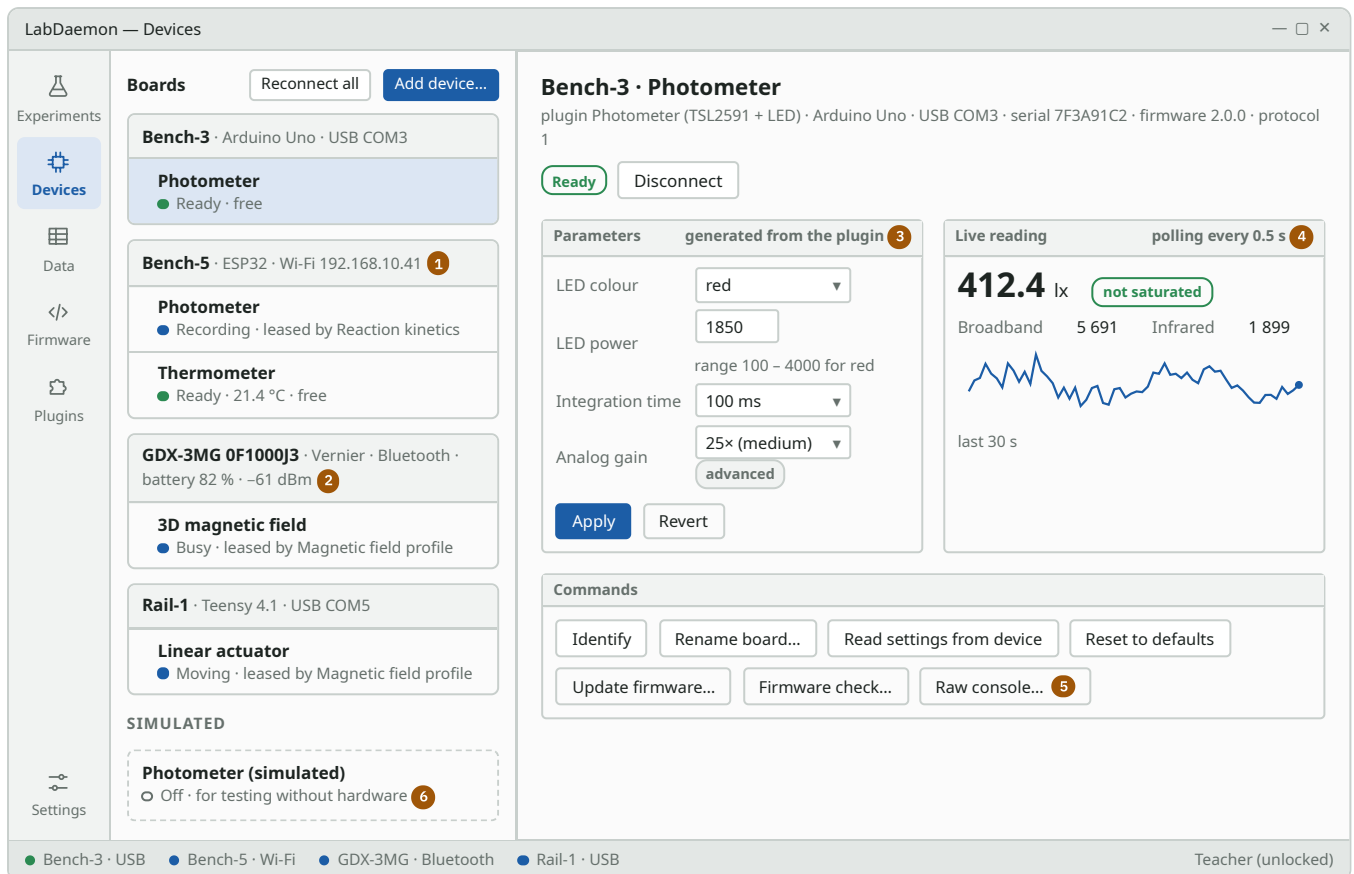
8 Mockups

These are low-fidelity in intent but use realistic content: measured values, units and file names are plausible for the real setups. They are drawn in the Fusion style that the app will use. Horizontal scrolling is expected on narrow screens.



Mockup 1·Experiment launcher

- 1 Categories come from each plugin's `category` ; search matches names and descriptions.
- 2 The card says which mode it will open in, following the rules in §6.
- 3 Requirements are matched against connected devices by capability. When one is missing, the card offers to connect it instead of failing later.
- 4 Folder plugins are marked. A broken plugin stays visible with its error instead of disappearing.
- 5 The status bar always shows every board with its link, and whether the student lock is on.



Mockup 2 · Devices

- 1 Boards are listed with their functions. Bench-5 is a combined ESP32: its photometer is recording for Reaction kinetics while its thermometer is free for another experiment (§7.2).
- 2 Each board shows its link. Bluetooth devices also show battery level and signal strength; a Vernier sensor is a board with a single function.
- 3 The form is built from `Parameter`s. The power range follows the selected colour. Advanced parameters are hidden while the student lock is on.
- 4 Live polling runs on the board's worker thread, interleaved with commands (§4.5). Watching needs no lease.
- 5 The raw console sends protocol commands by hand, completes verbs from `CMDS?`, and shows `#` debug lines and events. "Firmware check" runs the conformance suite (§4.4). Both are for teachers only.
- 6 Every device plugin ships a simulated twin, so labs can be prepared and rehearsed at home.

Add device

All links
USB
Wi-Fi
Bluetooth
Searching USB ports, mDNS and Bluetooth... 8 found
Search again

Name	Board	Functions	Found on	Status
Bench-3	Arduino Uno	photometer	USB COM3	connected
Bench-5	ESP32	photometer, thermometer	Wi-Fi · Bluetooth -58 dBm	via Wi-Fi
Bench-7	ESP32	photometer	USB COM8 · Bluetooth -70 dBm	new
Rail-1	Teensy 4.1	linear-actuator	USB COM5	connected
Cart-2	STM32 Nucleo-F446RE	motion-cart	USB COM9	no plugin
GDX-3MG 0F1000J3	Vernier	3D magnetic field	Bluetooth -61 dBm · 82 %	connected
GDX-TMP 0H1005K7	Vernier	temperature	USB	new
COM1	Communications Port	—	USB	no reply to ID?

Board on another network? Add it by IP address

Add

Bench-7

ESP32 · firmware 1.2.0 · serial 3C71BF5A1F88 · 1 function: photometer

Connect via

☒ Bluetooth · -70 dBm · LabInt-Bench-7
☐ USB · COM8
☐ Wi-Fi · not set up yet
Set up Wi-Fi...

Access token
read over USB

If unavailable
try USB, then Wi-Fi

☒ Remember this board and reconnect when LabDaemon starts

Cancel
Connect

Mockup 3 · Adding a device and choosing its link

- Discovery runs on every link at once: ID? on each serial port, the _labint._tcp mDNS service, and a Bluetooth scan for LabInt boards and Vernier sensors. A board found on several links is listed once, by serial number.
- The user chooses the link, and the choice and its fallback order are saved with the board. The same dialog chooses between USB and Bluetooth for Vernier sensors.
- Wi-Fi setup is offered while the board is also on USB. It sends the network credentials and reads the access token over the cable, so no secret travels over the air.
- A function nobody has written a plugin for yet is shown with its model name, not hidden. That is the hint to create one with labdaemon plugin new .
- Boards on another network than the PC can't be found by mDNS. The user types their IP address, and it is saved with the board.

Absorbance and concentration · session "CuSO₄ 3B" · autosaved 14:41

Guided Expert 1

- ✓ Devices
- ✓ Light setup
- ✓ Prepare the blank
- ✓ Measure the blank
- 5 Standard solutions**
- 6 Unknown sample
- 7 Results

Student lock on
Teacher unlock: Ctrl+Shift+T

Measure each standard solution

Rinse the cuvette with the next standard, fill it to the line and insert it with the clear faces towards the light. Type its concentration, then press **Measure** or Space. At least 3 standards; 5 recommended.

Standards					4 of 5 measured 2
#	c (mol/L)	I (lx)	T	A	
blank	0.000	412.4	1.000	0.000	measured
1	0.020	300.1	0.728	0.138	measured
2	0.040	222.5	0.540	0.268	measured
3	0.060	160.8	0.390	0.409	measured
4	0.080	—	—	—	<button>Measure</button>
5	type a concentration...				

● Live: 118.1 lx · stable for 3 s 3

Calibration curve

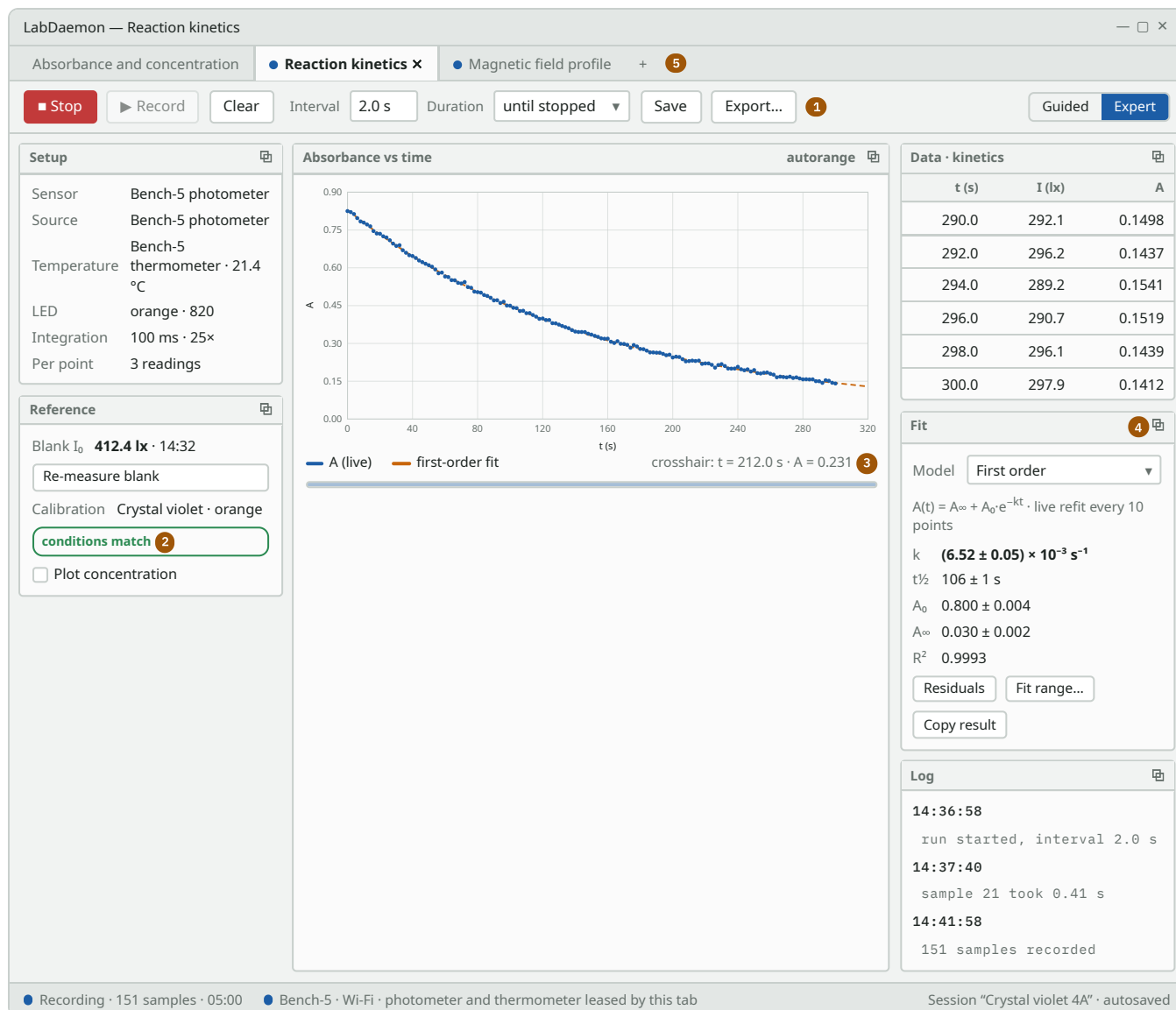
● measured — fit

$A = 6.785 \pm 0.071 \cdot c + 0.000 \quad R^2 \quad 0.9998$

$\epsilon \quad 6.79 \text{ L} \cdot \text{mol}^{-1} \cdot \text{cm}^{-1} (l = 1 \text{ cm})$

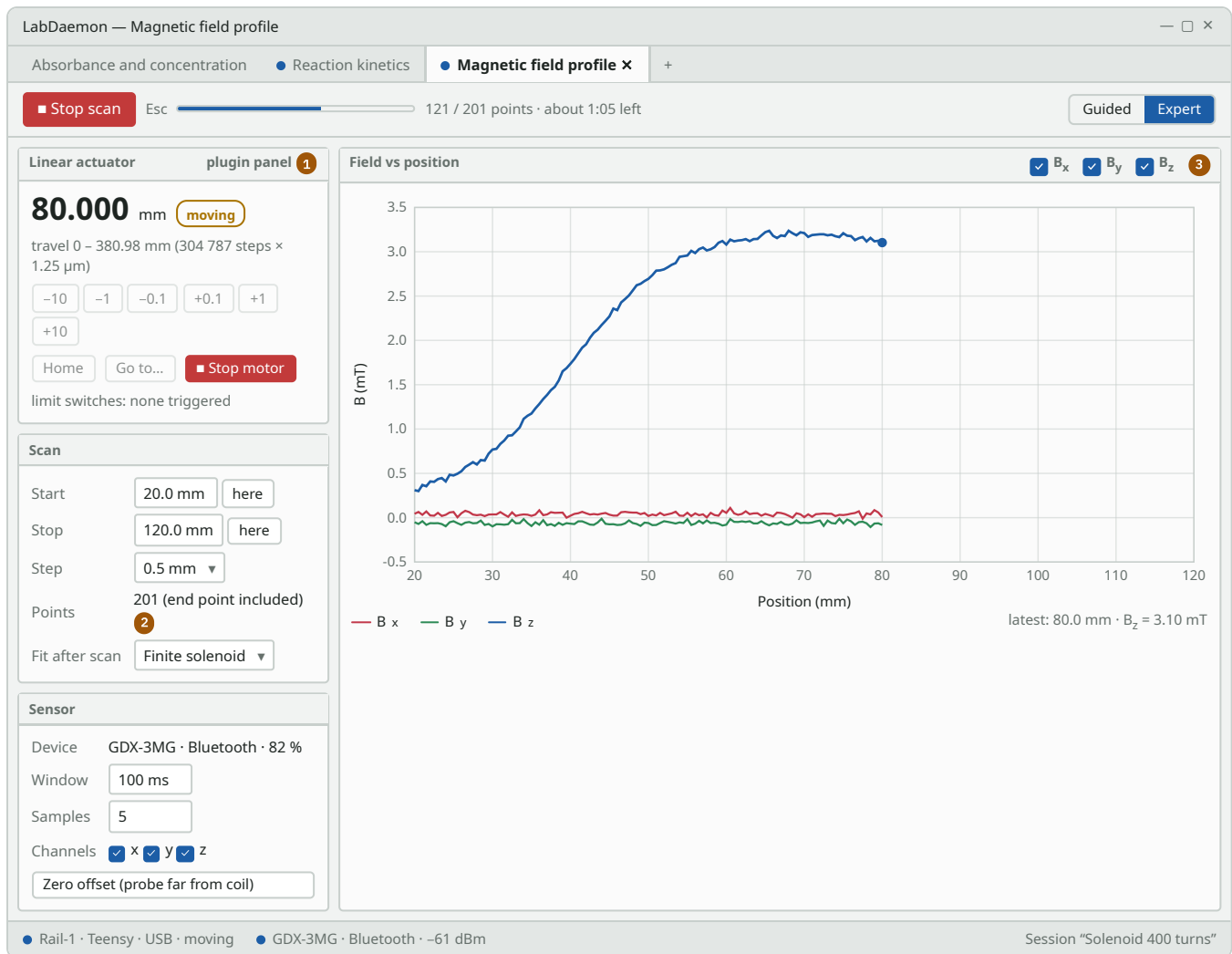
← Back 3 standards measured · the minimum is reached 4 Next: unknown sample →

- 1 The Guided/Expert switch keeps the same session. Switching here opens the expert layout (like mockup 5) with these four points already in it. With the student lock on, as here, the switch is hidden.
- 2 This is a `TableStep`: typing a concentration and pressing Measure runs `measure_standard(c)` on the runner thread. The blank row is included in the fit.
- 3 A live stability hint helps students wait for the reading to settle before measuring.
- 4 Completion rules come from the step (`min_rows=3`). "Next" is enabled as soon as the rule is met.



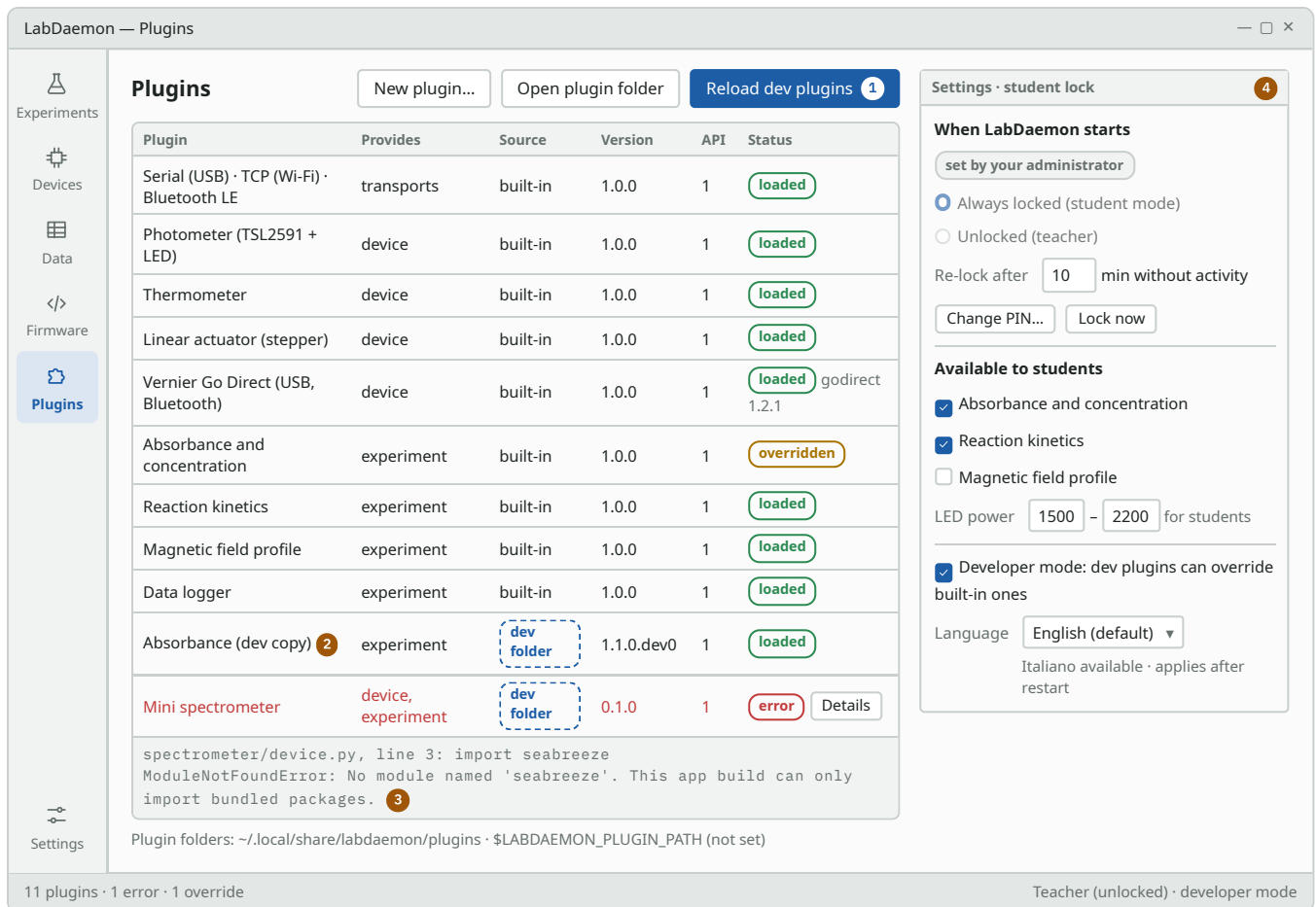
Mockup 5 · Expert mode, reaction kinetics

- 1 The run controls are generated from the experiment's `@run` signature (`interval_s`, `duration_s`). Esc stops the run.
- 2 The calibration library checks that the stored conditions (LED, power, integration time, gain) match the current ones.
- 3 Docks can be moved, floated (☐) and closed. The layout is remembered per experiment.
- 4 The Fit dock offers the models the experiment allows (first order, second order, linear...). Uncertainties come from the scipy covariance. The fit range can be dragged on the plot.
- 5 Experiments are tabs, and two are recording at once here: kinetics on Bench-5 and the field scan on Rail-1 and the Vernier sensor. Each tab holds leases on its own devices. The temperature channel comes from the second function of the same combined board.



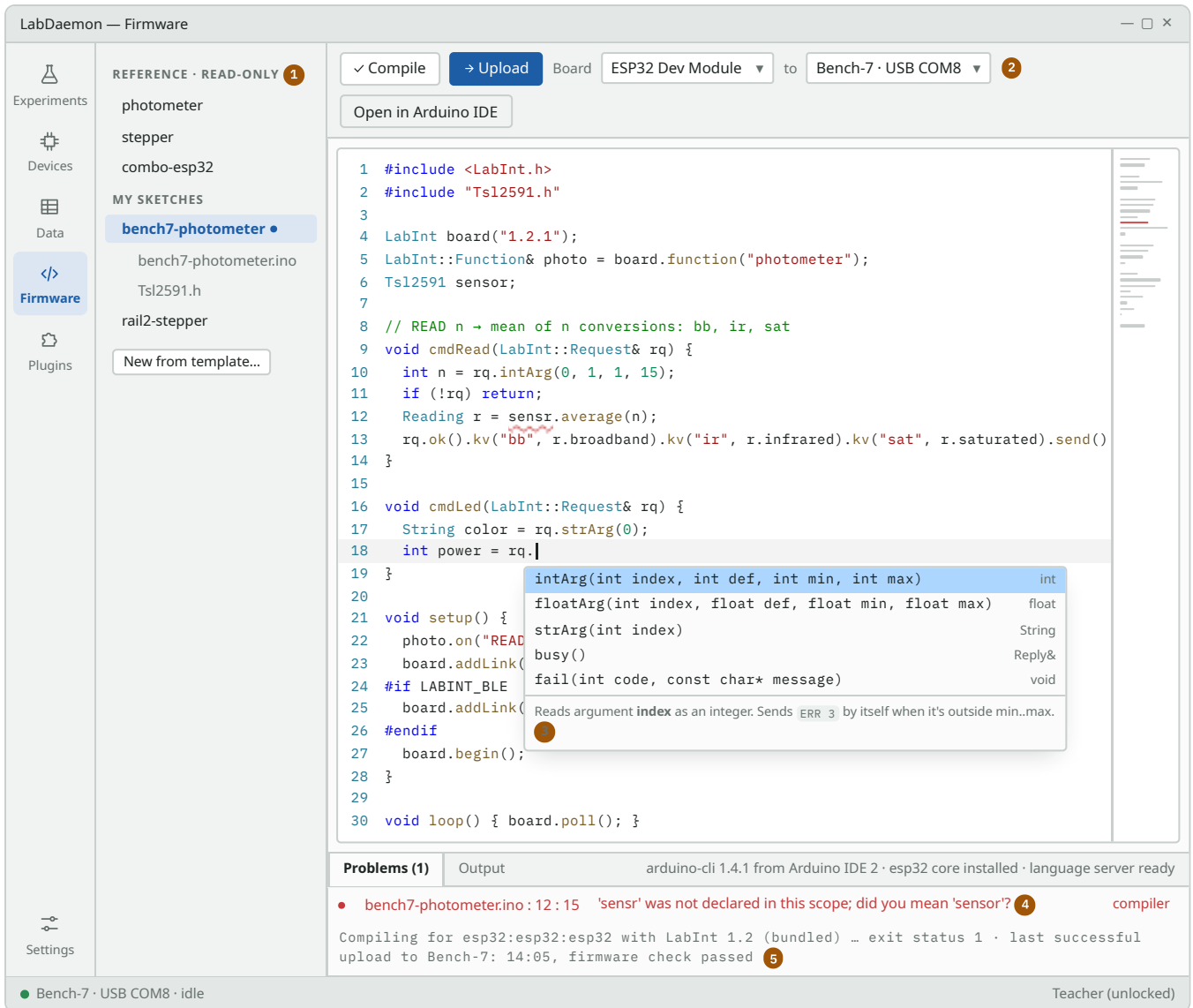
Mockup 6 · Expert mode, magnetic field profile

- 1 The stepper plugin contributes its own jog panel through `gui = "...:StepperPanel"`. The jog buttons are disabled while a scan owns the actuator. *Stop scan*, *Stop motor* and *Esc* all send `STOP` immediately; the motor halts mid-move and the points already measured are kept (§4.5). Limit-switch events appear here and in the log.
- 2 The start, stop and step values are experiment parameters with defaults, so a scan can no longer fail on an unset range. After the scan, the chosen model (finite solenoid, dipole) is fitted and drawn on the plot.
- 3 All three components are plotted in one view with toggles. Today only B_z ever shows, and the lowest points are clipped.



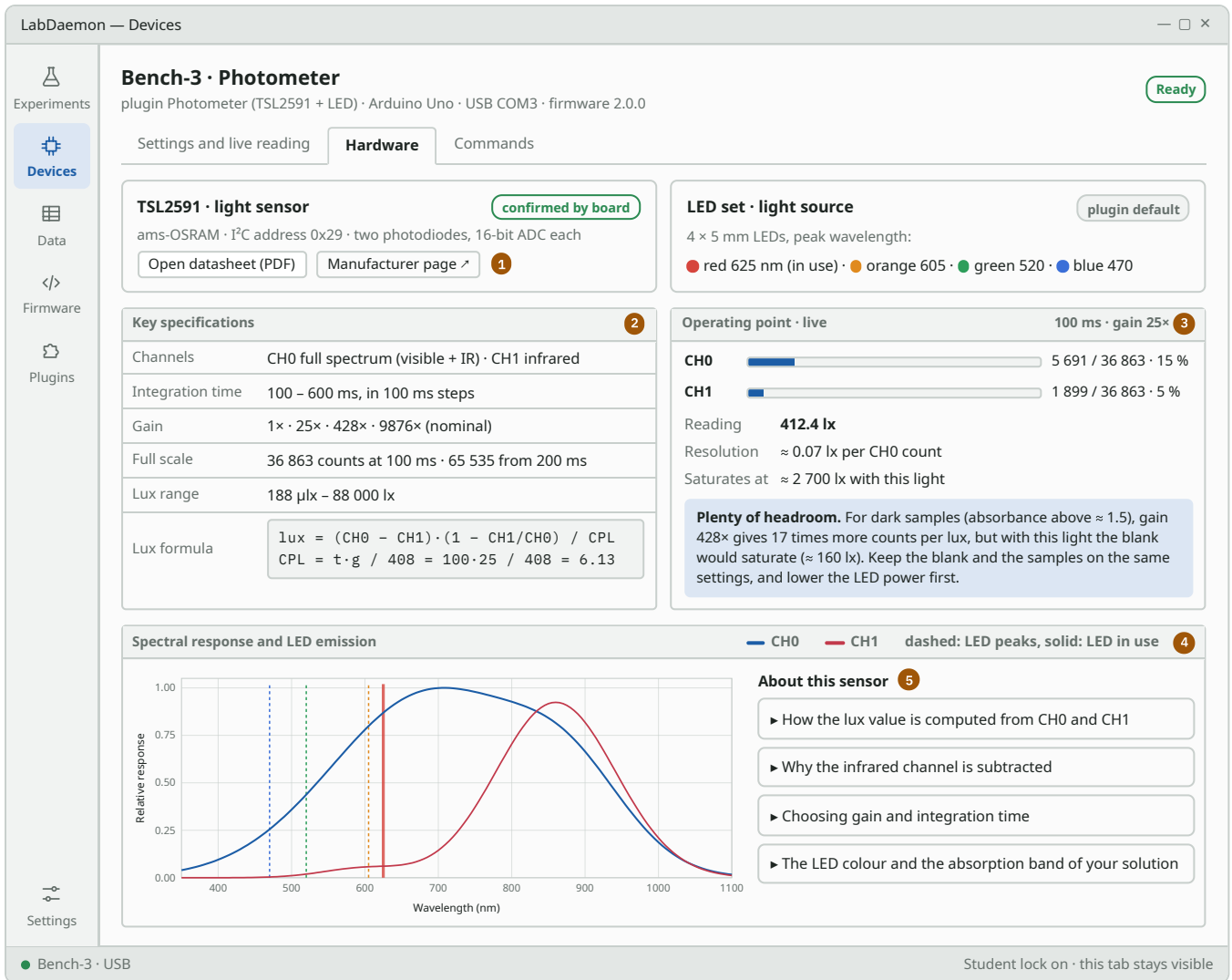
Mockup 7 · Plugins and student-lock settings

- 1 Reloading re-imports only the folder plugins that are not in use.
- 2 In developer mode a dev copy overrides the built-in plugin, and the built-in one is marked "overridden".
- 3 Errors say what went wrong and what the constraint is. In the installed app, folder plugins can only import packages bundled with it (§11).
- 4 Student-lock settings from §6.1, visible only when unlocked. "When LabDaemon starts" is fixed by the machine policy of this central install, so it shows as set by the administrator. The per-experiment switches and parameter limits are the teacher's.



Mockup 8 · Firmware view

- 1 Reference sketches ship with each release and are read-only. Editable copies live in `Documents/LabDaemon/firmware`. "New from template" also creates the matching plugin skeleton.
- 2 The board type comes from the connected board's ID? reply and can be overridden. Uploads go to the chosen board over USB.
- 3 Monaco, embedded with `qtmonaco`. Completion, hover help and live errors come from `arduino-language-server` and `clangd`, the same tools Arduino IDE 2 uses. Here they complete the LabInt library's own API.
- 4 Compiler errors are listed in Problems and marked in the editor (line 12) and the minimap. Clicking one jumps to the line.
- 5 Upload releases the board, flashes it, reconnects, checks the new firmware version and runs the firmware check (§7.3).



Mockup 9 · Devices, Hardware tab

- Parts come from the plugin's `hardware.toml`. The board confirms them through `HW?`. This board's firmware reports only the sensor, so the LED set shows as the plugin default. The datasheet opens in the built-in PDF viewer when a copy is available, otherwise on the manufacturer's site.
- Key figures, including the exact formula and constants the plugin uses. Here CPL is worked out for the current settings.
- The operating point is computed live from the current settings and readings, using the same `full_scale_counts` the saturation check uses. The hint explains what a setting change would do.
- The spectral response with the LED peaks shows what each colour actually measures. The curve shapes in this mockup are illustrative; the plugin ships curves digitised from the datasheet.
- Short notes from `notes.en.md` / `notes.it.md`. Wizard steps can link straight to one of them.

9 Porting the current code

CURRENT MODULE	BECOMES	WHAT CHANGES
<code>interfaces/microcontroller.py</code>	<code>transports/serial.py</code> , <code>transports/wire.py</code>	The LabInt wire protocol replaces one-letter commands and <code>executed</code> : deadlines, typed replies, exceptions instead of <code>print</code> .
Current Arduino sketches (not in this repo)	<code>firmware/sketches/photometer</code> , <code>.../stepper</code> on the LabInt library	Same hardware logic, new protocol; asynchronous moves with <code>STOP</code> ; the boards are reflashed.
<code>photometer/photometer.py</code>	<code>devices/tsl2591/</code>	TSL2591 lux formula kept; unused TSL2561 path dropped; raises instead of returning <code>-1</code> .
<code>photometer/configuration.py</code>	generated <code>ParameterForm</code>	The colour-to-power-range link is declared on the parameter.
<code>photometer/calibration.py</code>	<code>experiments/absorbance</code> + <code>core/calibration.py</code> + <code>core/analysis.py</code>	numpy fit with errors, blank included, reusable calibrations.
<code>photometer/measure.py</code>	<code>experiments/absorbance</code> (unknown sample), <code>experiments/kinetics</code>	The runner thread replaces an asyncio task that blocked the UI loop.
<code>linear_actuator/*</code>	<code>devices/stepper/</code> + its <code>gui.py</code>	Commands keyed by id; interruptible moves.
<code>interfaces/vernier/data_interface.py</code>	<code>devices/vernier/</code>	Device chosen explicitly; channels by key; USB or Bluetooth as a connection setting.
<code>magneticfield/*</code>	<code>experiments/field_profile/</code>	The scan is a <code>@run</code> with stop; all components plotted.
<code>setup/app_setup.py</code> (module-level global)	<code>core/manager.py</code> + <code>Session</code>	Passed explicitly; no shared global state.
<code>utils/interface_widgets.py</code> , <code>utils/debug.py</code>	<code>gui/widgets/</code>	Qt widgets.
<code>setup/logger_utils.py</code>	<code>gui/app.py</code> logging setup + <code>core/settings.py</code>	Log folder from <code>platformdirs</code> instead of a hard-coded <code>/tmp</code> .

Bugs from the code review that the design removes

BUG TODAY	REMOVED BY
Serial read loops forever when the device goes silent	The 500 ms reply rule, <code>BUSY</code> with a heartbeat, and deadlines in <code>WireClient</code> (§4.4)
"Motor home" button sends end-home	Commands keyed by id (§4.3)
<code>log10</code> of a negative number when light is too dim	Typed <code>LightReading</code> ; <code>analysis.absorbance</code> raises a clear <code>ExperimentError</code>
Reset Ranges crashes; scanning before setting a range crashes	Typed experiment parameters with defaults (§4.2)
Regression filters molarity and absorbance on different columns	Fits run on dataset rows; the blank is included (§4.7)
Loading a calibration resets the LED power	<code>Calibration.apply()</code> sets conditions atomically

BUG TODAY	REMOVED BY
Saturation is checked against the TSL2561's 65 000 counts; at 100 ms the TSL2591 saturates at 36 863, so it is never detected	<code>full_scale_counts</code> from the hardware sheet, per integration time (§7.4)
<code>bool("0") == True</code> in the calibration check	Typed <code>key=value</code> replies (<code>LIMITS?</code> → <code>homed=1</code> , read with <code>Reply.bool</code>)
A field scan can't be stopped while the stepper moves	Asynchronous <code>MOVE</code> with <code>STOP</code> (§4.5)
Only B_z ever shown; low points clipped	<code>LivePlot</code> with toggles and padded auto-range
Widgets showing different values from those used	<code>ParameterSet</code> as the single source of truth
Monotonic clock in the <code>t(s)</code> column; <code>:</code> in file names	Relative time plus an ISO start time; safe file names

10 Using the core from Jupyter

Install LabDaemon into the notebook's environment from the repo or a release wheel, with the `[jupyter]` extra (§11.3). The `Lab` facade uses the same registry, workers and experiments as the GUI, and never imports Qt. Calls block and return typed results, and sessions saved here open in the desktop app.

```
from labdaemon import Lab

lab = Lab()                                     # discovers plugins, including folder plugins
lab.scan()                                     # all links: [Board('Bench-3', usb='COM3', functions=['photometer']),
                                              #   Board('Bench-5', wifi='192.168.10.41', ble='LabInt-Bench-5',
functions=['photometer', 'thermometer']), ...]
photo = lab.connect(port="COM3")["photometer"] # USB; one worker per board, proxies per function
bench5 = lab.connect(host="bench-5.local", token="K7Q2M9XD") # Wi-Fi
thermo = bench5["thermometer"]
mag = lab.connect(vernier="GDX-3MG 0F1000J3", link="ble")["magnetic"]
# photo = lab.simulate("tsl2591_photometer")    # same API, no hardware

photo.set_output("red", 1850)
photo.read_light(samples=3) # LightReading(lux=412.35, broadband=5691, infrared=1899, saturated=False)

exp = lab.experiment("absorbance", sensor=photo, source=photo)
exp.measure_blank()
for c in (0.02, 0.04, 0.06, 0.08):
    input(f"Insert the {c} mol/L standard and press Enter")
    exp.measure_standard(c)
print(exp.calibration.fit.formula(), exp.calibration.fit.params["m"]) # A = 6.785·c + 0.000 6.785 ±
0.071 L/mol

run = lab.experiment("kinetics", sensor=photo, source=photo).start("record", interval_s=2)
run.stop(); run.wait() # long runs execute on the experiment's runner thread

df = exp.session.dataset("standards").to_pandas() # optional [jupyter] extra
exp.session.save("cuso4.labint")
lab.close()
```

11 Windows and Linux: installation and platform details

11.1 Installing for all users or per user

DECISION The same build installs either centrally, for every account on a lab PC, or for one user without admin rights.

	FOR ALL USERS (CENTRAL)	PER USER
Windows	The Inno Setup installer in admin mode installs to <code>C:\Program Files\LabDaemon</code> with a Start-menu entry for everyone. Silent for Intune, SCCM, PDQ or a GPO script: <code>LabDaemon-Setup.exe /ALLUSERS /VERYSILENT /NORESTART</code> . A silent EXE is enough for now. An MSI (WiX) can be added later around the same PyInstaller output, without changing the app.	The same installer with "Only for me" installs to <code>%LOCALAPPDATA%\Programs\LabDaemon</code> without admin rights. Silent: <code>/CURRENTUSER /VERYSILENT</code> .
Linux	A <code>.deb</code> for Debian/Ubuntu lab PCs installs to <code>/opt/labdaemon</code> , with a desktop entry and udev rules for Vernier and USB-serial boards.	An AppImage in <code>~/Applications</code> , integrated into the desktop on first run. The udev rules need a one-time admin step, which the user guide documents.
Updates	IT redeploys the new version.	An optional check against GitHub releases at startup. The machine policy can turn it off.

11.2 Machine policy

A central install can include an admin-writable policy file: `%PROGRAMDATA%\LabDaemon\policy.toml` or `/etc/labdaemon/policy.toml`. Values set there override user settings, and Settings shows them locked with "Set by your administrator". Settings resolve in this order: policy, then user settings, then defaults.

```
[student]
mode = "forced"                                # forced | default | off
pin_hash = "sCrypt$16384$8$1$..."
relock_minutes = 10

[paths]
sessions = '\\fileserver\lab\sessions'          # class folder
shared_calibrations = '\\fileserver\lab\calibrations' # everyone reads, unlocked teachers write

[plugins]
allow_user_plugins = false                      # only the machine-wide folder
developer_mode = false

[links]
wifi = true
bluetooth = true

[firmware]
arduino_cli = 'C:\Program Files\Arduino IDE\resources\app\lib\backend\resources\arduino-cli.exe'
data_dir = 'C:\ProgramData\LabDaemon\arduino'  # shared board cores and tools

[app]
language = "en"
update_check = false
```

Platform details

- Config, data (calibrations, plugins, recovery files) and log folders come from `platformdirs`, plus the machine-wide folders above.
- Boards are remembered by serial number, not by COM number, IP address or Bluetooth address.
- USB on Linux: a `PermissionError` produces the `dialout / uucp` group hint. On Windows, some Arduino clones need the CH340 driver; the Devices view links to it when no port shows up.
- Bluetooth: Windows 10 or later, or BlueZ on Linux, and a Bluetooth LE adapter. One scanner serves both Vernier sensors and ESP32 boards.
- Wi-Fi: lab PCs reach the boards directly. mDNS works only within the PC's own network, so boards elsewhere are added by IP address. The central installer adds a Windows firewall rule for mDNS (UDP 5353), which a per-user install asks for the first time it searches.

Packaging

- Development: `uv run labdaemon`.
- One PyInstaller one-folder build per platform feeds all the installers. It must run with `--copy-metadata labdaemon`, otherwise entry points do not resolve in the frozen app.
- In the installed app, folder plugins can use only the libraries bundled with it. The Plugins view lists them.
- `scipy` adds roughly 40 MB. Excluding submodules the app doesn't use (`scipy.io`, `scipy.signal`, ...) keeps that down; the real saving will be measured in M4.

Firmware tools

- In the app: the Firmware view (§7.3) uses `arduino-cli`, found inside Arduino IDE 2, on the PATH, or at a configured path.
- For development and CI: PlatformIO builds every reference sketch for every board, for example `pio run -e esp32dev -t upload`.
- Each release ships precompiled reference firmware for AVR and ESP32, with `avrdude` and `esptool`, so "Update firmware..." works without any compiler installed.
- The `LabInt` library is also published as a zip for the Arduino IDE.

11.3 Python version and distribution

- **Python 3.14 or newer** (`requires-python = ">=3.14"`; `.python-version` moves from 3.13 to 3.14). The installers bundle their own Python, so lab PCs need nothing installed beforehand.
- **The standard build, with the GIL.** The worker threads spend their time waiting for I/O, where the GIL is released, so the free-threaded build would gain nothing here and not every dependency supports it yet. Nothing in the design stops a later switch.
- **Not on PyPI.** Each release publishes the installers and a wheel on the project's GitHub releases. For Jupyter or scripts, install into that environment from a checkout (`uv pip install -e ".[jupyter]"`) or from the release wheel.

DEPENDENCIES	PACKAGES
Runtime	PySide6-Essentials , PySide6-Addons (web engine, loaded only by the Firmware view), qtmonaco , esptool , pyqtgraph , numpy , scipy , pyserial , bleak (Bluetooth LE), zeroconf (mDNS), platformdirs , loguru , babel
Extras (installs from the repo or a release wheel; the installers include everything)	[vernier] godirect with USB and Bluetooth · [jupyter] pandas · [xlsx] openpyxl
Development	pytest , pytest-qt , ruff , pyright , pyinstaller ; for firmware: platformio
Removed	ipywidgets , ipysheet , ipympl , jupyter , scikit-learn ,.gdx

12 Testing and quality

- **Protocol tests** run `WireClient` against the `WireSimulator` and scripted transcripts: normal replies, `ERR` codes, events arriving between replies, `#` debug lines, a silent board (must time out), `STOP` during `BUSY`, a board that reboots mid-session, and a port that disappears.
- **Links:** the TCP transport is tested against a `WireSimulator` served on localhost, including mDNS records and `AUTH`. The BLE transport is tested with a fake bleak client that splits lines across notifications. The same board seen on two links must be listed once.
- **Firmware:** CI compiles every sketch for every board environment (AVR, ESP32, STM32, Teensy) and runs the parser unit tests in the PlatformIO `native` environment. `firmware check` runs the conformance suite against real boards.
- **Leases and policy:** take-over and refusal rules, per-function leases on combined boards, and policy values overriding user settings.
- **Translation check:** CI fails when a built-in string has no Italian translation (§7.1).
- **Device and experiment tests** run against simulated devices. For example, the simulated photometer follows Beer–Lambert, and the absorbance test checks that the fitted slope recovers ϵ .
- **Plugin conformance:** one parametrized test runs every registered plugin through the same checks as `plugin check`.
- **Architecture test:** importing `labdaemon.core` with PySide6 blocked must succeed (rule R1).
- **GUI smoke tests** (pytest-qt): open every experiment in both modes with simulated devices, run one action, and save.
- **Hardware tests** are marked `@pytest.mark.hardware` and skipped by default.
- **CI:** GitHub Actions on Ubuntu and Windows, Python 3.14, running ruff, pyright and pytest. Installers and the wheel are built on release tags.

13 Roadmap

M0

Skeleton and protocol. Core contracts, registry, events, boards and functions, `BoardWorker`, `WireClient`, and `PROTOCOL.md` v1 with its Python `WireSimulator`. Settings and the policy merge. A simulated photometer and the main window with the Devices view, in English with the translation pipeline already wired.

Done when a simulated photometer connects over the simulated wire protocol and its live reading updates in the status bar and the Devices view.

M1

Photometer parity. The `LabInt` C++ library on AVR, photometer firmware 2.0 and reflashing the boards, `firmware check`, the TSL2591 plugin with its hardware sheet, the absorbance experiment in both modes, the calibration library, linear fits with uncertainties, and `.labint` sessions.

Done when a complete absorbance lab runs on real hardware over USB on Windows and on Linux.

M2

Kinetics, data and leases. The reaction kinetics experiment with first- and second-order fits, the Fit dock, the Data view, CSV presets, experiment tabs with leases, the student lock, and the Hardware tab with its live operating point.

M3

Field profile. Stepper firmware on Teensy with asynchronous `MOVE`, `STOP` and limit events; the stepper plugin; the Vernier plugin over USB and Bluetooth; the field profile experiment with immediate stop and solenoid and dipole fits.

M4

Release. Windows installer (all users or per user, silent switches), `.deb` and AppImage, the machine policy, the complete Italian translation, the user guide in English and Italian, and "Update firmware..." with precompiled reference firmware (level 1 of §7.3).

M5

Wireless and more boards. ESP32 links (Wi-Fi with mDNS and `AUTH`, Bluetooth with the Nordic UART Service), the Wi-Fi setup dialog, a combined-board reference sketch (photometer + thermometer), and STM32 builds of the library.

M6

Firmware view and extensibility. Compile and upload through `arduino-cli`, the Monaco editor with the Arduino language server, sketch and plugin templates, the plugin `new` / `check` commands, the plugin author guide and the generic data logger. Optionally, an MSI package.

14 Open questions and early checks

No design questions are open. These points should be confirmed in M0, before much code depends on them:

C1 Python 3.14 wheels

Install PySide6-Essentials, pyqtgraph, numpy, scipy, pyserial, bleak, zeroconf and godirect on Python 3.14, on Windows and on Linux, and build a PyInstaller smoke test.

C2 Vernier over Bluetooth

Connect a Go Direct sensor through godirect's Bluetooth support on both platforms, and check that it can share the Bluetooth adapter with bleak scanning for ESP32 boards.

C3 Memory on the Uno

Measure how much of the Uno's 2 KB of RAM the LabInt library and the photometer sketch use together. If the margin is thin, the combined-board and event features get compile-time switches.

C4 Wi-Fi in the lab

With one ESP32: mDNS discovery on the lab PCs' network, connecting by IP from another network, and the Windows firewall prompt in a per-user install.

C5 Monaco and the Arduino language server

Run `qtmonaco` on PySide6 with `arduino-language-server` and `clangd` from Arduino IDE 2, on an ESP32 sketch that uses LabInt, on Windows and on Linux. Check completion, hover help and live errors through the WebSocket bridge. Not strictly needed in M0, but it should be settled before M6 is planned in detail.

C6 Installer size

Measure how much the embedded web engine (`PySide6-Addons`) adds to the Windows installer and the AppImage. If it is too much for school PCs, the Firmware view's editor can become a separate optional download.

Answered questions. Draft 0.2: Q1 firmware, Q2 stopping, Q4 languages, Q6 scipy. Draft 0.3: Q3, Q5, Q7–Q13. Draft 0.4: Q14 Wi-Fi addressing, Q15 package name, Q16 installer. All of them are in the decisions table in §0.

